UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CARRERA DE CIENCIA DE LA COMPUTACIÓN



Modelo híbrido HyperNCD-Sonata para el descubrimiento de clases novedosas en la segmentación semántica de nubes de puntos de patrimonio arquitectónico

TRABAJO DE INVESTIGACIÓN

Para optar el grado de Bachilleren Ciencia de la Computación

AUTOR(ES)

Renzo Felix Aponte

Manyory Cueva Mendoza

ASESOR(ES)

Fiestas Iquira, Jose Antonio

Lima - Perú

2026

Dedicatoria:

A nuestras familias, por el apoyo constante durante toda la carrera.

Agradecimientos:

A la Facultad de Ingeniería Civil de UTEC, por facilitar el levantamiento con escáner láser terrestre de la Catedral de Lima que constituye el objeto de estudio de este trabajo. Al equipo del clúster de cómputo Khipu, por el acceso a los recursos de GPU necesarios para la experimentación. A nuestro asesor, por la orientación en la definición del alcance y en el diseño experimental.

TABLA DE CONTENIDO

Pág.

RESUMEN 1
ABSTRACT 3
INTRODUCCIÓN 4
Formulación del problema 5
Objetivos de investigación 6
Objetivo General 6
Objetivos Específicos 6
Justificación 7
Alcance y limitaciones 8

CAPÍTULO I REVISIÓN CRÍTICA DE LA LITERATURA 11
1.1 HyperNCD 11
1.2 Sonata 13
1.3 Point Transformer V3 15
1.4 BRDL 17
1.5 Conclusión: aportes del proyecto 19

CAPÍTULO II MARCO TEÓRICO 24
2.1 Ingeniería Civil y Conservación del Patrimonio 24
2.1.1 Nube de Puntos 24
2.1.2 Levantamiento Láser Terrestre (TLS) 25
2.1.3 Formato LAS 25
2.1.4 Centrado de Coordenadas 26
2.1.5 Vecindad de los q Vecinos Más Cercanos 27

2.1.6 Filtrado Estadístico de Valores Atípicos (SOR) . . . 27
2.1.7 Vóxel y Voxelización . . . 28
2.1.8 Normal de Superficie y su Estimación . . . 28
2.1.9 Sección Espacial . . . 30
2.1.10 Segmentación Semántica . . . 30
2.1.11 Clase Conocida y Clase Novedosa . . . 30
2.2 Computación . . . 31
2.2.1 Descriptor de Punto . . . 31
2.2.1.1 Descriptor Semántico . . . 31
2.2.1.2 Descriptor Geométrico Analítico . . . 32
2.2.1.3 Descriptor Auto-supervisado . . . 32
2.2.2 Aprendizaje Auto-supervisado . . . 33
2.2.3 Codificador Congelado . . . 33
2.2.4 Atajo Geométrico . . . 33
2.2.5 Serialización . . . 33
2.2.6 Rejilla Dispersa y Tensor Disperso . . . 34
2.2.7 Convolución Dispersa . . . 34
2.2.8 MinkowskiUNet-34C . . . 35
2.2.9 Muestreo por Rejilla e Índice Inverso . . . 35
2.2.10 Propagación Inversa del Submuestreo . . . 36
2.2.11 Caja de Sección y Corte . . . 36
2.2.12 Reconstrucción de Superficie de Poisson . . . 36
2.3 Algoritmos . . . 36
2.3.1 Descubrimiento de Clases Novedosas (NCD) . . . 37
2.3.2 Región o Superpunto . . . 37
2.3.3 Prototipo . . . 37
2.3.4 Hipergrafo . . . 38
2.3.4.1 Hiperarista . . . 38

2.3.4.2 Afinidad entre Prototipos 39
2.3.5 Pseudo-etiqueta 39
2.3.6 Normalización de Sinkhorn-Knopp 40
2.3.7 Arquitectura de HyperNCD 40
2.3.8 Arquitectura de Sonata 41
2.4 Métricas de Evaluación de Modelos 42
2.4.1 Intersection over Union (IoU) 42
2.4.2 IoU Media (mIoU) 42
2.4.3 Análisis de Dominancia 42
2.4.4 Emparejamiento Húngaro 43
2.4.5 Índice de Rand Ajustado (ARI) 43
2.4.6 Información Mutua Normalizada (NMI) 43
2.4.7 Linear Probing 44
2.4.8 Ganancia Absoluta 44
2.4.9 Brecha Cerrada 44
2.4.10 Atribución por Etapas 45
2.5 Diseño de la Tesis 45
2.5.1 Módulo 1: Preprocesamiento 45
2.5.2 Módulo 2: Semántica 47
2.5.3 Módulo 3: *Encoder* 47
2.5.4 Módulo 4: Fusión 48
2.5.5 Módulo 5: Salida 48
2.5.6 Diseño Propuesto e Implementación Actual 49

CAPÍTULO III MARCO METODOLÓGICO **50**

3.1 Diagrama de Flujo General 50
3.2 Hardware y Repositorio 51
3.3 Diseño General del Sistema 52
3.3.1 Estructura y Contrato de los Módulos 52

3.3.2 Herramientas por Módulo 53
3.3.3 Visor 3D del Proyecto 53
3.4 Unidad 1. Preprocesamiento 54
3.4.1 Datos de Entrada y Ejecución 54
3.4.2 Procedimiento 55
3.4.3 Verificación y Reproducibilidad 63
3.5 Unidad 2. Extracción de Descriptores y Fusión 64
3.5.1 Rama Semántica 64
3.5.2 Rama Auto-supervisada Congelada 64
3.5.3 Punto de Fusión 71
3.6 Unidad 3. Entrenamiento, Calibración y Evaluación 72
3.6.1 Procedimiento Experimental 72
3.6.2 Protocolo de Repetición 73
3.6.3 Trazabilidad de la Métrica 73
3.6.4 Verificación Empírica del Entorno de Cómputo 73
3.7 Riesgos y Mitigación 74
3.8 Conclusión del Marco Metodológico 74
ANEXOS 77

ÍNDICE DE TABLAS

Tabla A Cadena de reducción de datos del preprocesamiento 9

Tabla B Requisitos del método y recursos del clúster Khipu 9

Tabla 1.1 Publicaciones revisadas y su aporte a cada objetivo 11

Tabla 1.2 Ablación de HyperNCD en SemanticPOSS 12

Tabla 1.3 *Linear probing* sobre ScanNet 14

Tabla 1.4 Eficiencia de PTv3 frente a la red dispersa 16

Tabla 1.5 Ablación de BRDL en S3DIS 18

Tabla 1.6 Aportes del proyecto 22

Tabla 2.1 Archivos del sistema y archivos de inspección 47

Tabla 2.2 Implementación actual y aporte por módulo 49

Tabla 3.1 Contrato de los módulos 53

Tabla 3.2 Coordenadas de PL7 antes y después del centrado 58

Tabla 3.3 Metadatos de PL7 63

Tabla 3.4 Archivo de descriptores de PL13 69

Tabla 3.5 Observables de Sonata en PL13: esperado y medido 69

Tabla A.1 Herramientas por módulo 77

Tabla C.1 Parámetros del *encoder* de Sonata por etapa 83

ÍNDICE DE FIGURAS

Figura 2.1 Arquitectura de HyperNCD 40
Figura 2.2 Auto-destilación de Sonata 41
Figura 2.3 Diseño del modelo híbrido propuesto 46

Figura 3.1 Flujo general del proyecto 51
Figura 3.2 Flujo del preprocesamiento en PL7 56
Figura 3.3 PL7 tras la carga 57
Figura 3.4 Filtrado SOR 59
Figura 3.5 PL7 con los puntos descartados por el SOR en rojo 59
Figura 3.6 Voxelización 60
Figura 3.7 PL7 antes y después de la voxelización 60
Figura 3.8 Cubo de 1 m de PL7 antes y después de la voxelización 61
Figura 3.9 Estimación de normales 62
Figura 3.10 Normales de PL7 como color 62
Figura 3.11 Flujo de la extracción con Sonata en PL13 65
Figura 3.12 Matriz S de PL13, entrada de Sonata 66
Figura 3.13 Preparación de la nube para Sonata 67
Figura 3.14 Inferencia de Sonata 68
Figura 3.15 PL13 en sus tres etapas 68
Figura 3.16 Similitud y k-means de los descriptores de PL13 71
Figura C.1 Arquitectura del *encoder* de Sonata 82

RESUMEN

La documentación digital del patrimonio construido depende de levantamientos láser cuya interpretación sigue siendo manual. Este trabajo propone un modelo híbrido que sustituye el descriptor geométrico del mejor método de descubrimiento de clases novedosas por un codificador auto-supervisado, y mide la mejora sobre el levantamiento de la Catedral de Lima.

Un escáner láser registra dónde hay materia, pero no qué elemento representa cada punto. Los métodos de segmentación semántica tridimensional usan una lista cerrada de categorías y asignan todo elemento imprevisto a la clase más parecida. El objetivo es mejorar de forma medible el reconocimiento de los elementos sin etiqueta.

La investigación es aplicada, cuantitativa y de diseño experimental comparativo. Primero se diagnostica el mejor método de descubrimiento de clases novedosas reanalizando su ablación; luego se sustituye su componente más débil por un codificador auto-supervisado. La evaluación usa el levantamiento de la Catedral de Lima, dividido en secciones, contra una línea base de tres corridas.

El método base logra entre 11,8 y 47,5 puntos de intersección sobre unión en las clases no vistas, frente a 49,8 del supervisado. Su ablación da al descriptor geométrico solo 4,2 puntos sobre una base de 23,2, menos de la mitad de los 9,6 del hipergrafo. Eliminar esa dependencia eleva un descriptor congelado de 5,6 a 72,5 puntos. Se define además una medida de brecha cerrada, pues el sesgo hacia las clases mayoritarias cuesta hasta 15,34 puntos.

El desequilibrio del método base lo documentan sus propios autores y sigue sin corregirse, pero queda por establecer si persiste sin etiquetas. Cada punto de mejora es corrección manual que el especialista deja de hacer.

Palabras clave:

Nubes de puntos; Segmentación semántica tridimensional; Descubrimiento de clases novedosas; Aprendizaje auto-supervisado; Hipergrafos; Patrimonio arquitectónico

ABSTRACT

HyperNCD-Sonata hybrid model for novel class discovery in the semantic segmentation of architectural heritage point clouds

The digital recording of architectural heritage produces point clouds that show where matter is, but not which element each point stands for. Three-dimensional semantic segmentation methods work over a closed list of categories and assign every unforeseen element to the closest known class. The objective is to measurably improve the recognition of elements absent from the training labels.

This is applied, quantitative research with a comparative experimental design. We first diagnose the best available method for novel class discovery through its ablation study, and then replace its weakest component with a self-supervised encoder. The evaluation uses the survey of the Cathedral of Lima, divided into sections, against a three-run baseline.

The base method reaches between 11.8 and 47.5 points of intersection over union on the unseen categories, against 49.8 for the supervised equivalent. Its ablation credits the geometric descriptor with only 4.2 points over a base of 23.2, less than half of the 9.6 of the hypergraph. Removing that dependence on low-level signals raises a frozen descriptor from 5.6 to 72.5 points. We also define a closed-gap measure, since correcting the bias towards majority classes adds up to 15.34 points.

The imbalance is documented by the authors themselves and remains uncorrected, but whether it persists without labels for the target categories is still to be established. Every point gained is correction work the specialist no longer does.

Keywords:

Point clouds; Three-dimensional semantic segmentation; Novel class discovery; Self-supervised learning; Hypergraphs; Architectural heritage

INTRODUCCIÓN

Cuando se levanta una edificación patrimonial con un escáner láser terrestre, el resultado es una nube de puntos $P = \{(x_i, f_i)\}_{i=1}^{N}$, donde $x_i \in \mathbb{R}^3$ es la ubicación del punto y f_i los atributos que registra el escáner. En ingeniería civil ya son la fuente primaria para el control geométrico, el modelado como construido y la documentación de obra [1]. Lo que no aparece es la etiqueta. El archivo sabe dónde hay materia, pero no qué elemento es cada cosa, y esa carencia es la que bloquea el paso de la nube al modelo BIM y al informe estructural o de conservación [1]. Hoy ese trabajo se hace a mano, y la literatura de patrimonio lo documenta como el cuello de botella del proceso. La construcción del conjunto de referencia ArCH exigió anotar punto por punto diecisiete escenas de edificios históricos [2]. No existe un programa que automatice esa tarea sobre clases no catalogadas, y ese vacío es el que este proyecto busca cubrir.

La tarea no se resuelve con fotografías, porque un clasificador de imágenes depende del ángulo y de la iluminación, mientras que la nube de puntos guarda la geometría sin depender del punto de vista [3]. Sobre esa observación se construyeron los trabajos de patrimonio. Pierdicca et al. [3] adaptan una red de grafos dinámicos con normales y color, con buenos resultados en muros y techos pero caídas en clases minoritarias como columnas y arcos, y Matrone et al. [2] confirman ese desbalance sobre un mismo conjunto de edificios. El límite aparece con claridad en Grilli y Remondino [4]. Un modelo entrenado sobre un edificio pierde desempeño al aplicarse a otro de tipología distinta, porque los descriptores geométricos que usa no son estables entre construcciones. Una tesis previa de UTEC sobre este mismo tipo de edificio [5] llega al mismo resultado, ya que su modelo dejó de reconocer columnas y arcos al pasar de un patio a una capilla del mismo inmueble, y la única salida fue etiquetar a mano y reentrenar.

Los métodos generales de segmentación semántica en tres dimensiones tampoco alcanzan. PTv3 [6] llega a 77,5 mIoU y Sonata [7] a 79,4 sobre ScanNet (Ec. (II.17)), y otras arquitecturas como Stratified Transformer [8] o Superpoint Transformer [9] compiten en el mismo orden, pero todas trabajan con una lista de clases fijada antes de entrenar y asignan todo punto imprevisto a la clase conocida más parecida, sin avisar que no saben. En patrimonio eso es serio, porque cada edificio tiene elementos propios que no figuran en ningún catálogo previo [4], y el problema se agrava con las clases minoritarias, donde el sesgo hacia las mayoritarias llega a costar 15,34 puntos de mIoU [10].

El descubrimiento de clases novedosas, o NCD, elimina ese supuesto y permite trabajar con clases que el modelo nunca vio etiquetadas. Riz et al. [11] lo introdujeron para nubes de puntos y Xu et al. [12] lo refinaron con auto-etiquetado adaptativo. Su mejor método actual es HyperNCD [13], que resume la nube en *prototipos*, vectores alrededor de los cuales se agrupan los puntos parecidos, y los conecta mediante un *hipergrafo*, donde cada arista puede unir varios prototipos a la vez. Su problema es que rinde mal justo donde debería ser fuerte, entre 11,8 y 47,5 mIoU en las clases novedosas, frente a 49,8 del supervisado equivalente [13]. Este trabajo señala como causa probable el descriptor geométrico calculado a mano sobre el que arma sus prototipos, el tipo de señal que Sonata [7] considera perjudicial, y propone reemplazarlo por su codificador auto-supervisado congelado.

El Capítulo I presenta la revisión crítica de la literatura; el Capítulo II, el marco teórico y el análisis de arquitectura del que se deriva el punto de la sustitución; el Capítulo III, el marco metodológico; y el Capítulo IV, los resultados esperados.

Formulación del problema

Dado el levantamiento P de una edificación patrimonial, con un subconjunto

etiquetado de clases conocidas $\mathcal{C}_k$ y otro sin etiquetar de clases novedosas $\mathcal{C}_n$, donde $\mathcal{C}_k \cap \mathcal{C}_n = \emptyset$, el problema computacional consiste en producir una asignación $\hat{y} : P \rightarrow \mathcal{C}_k \cup \mathcal{C}_n$ que clasifique bien los puntos conocidos y descubra agrupaciones consistentes para los novedosos. El mejor método disponible alcanza solo entre 11,8 y 47,5 mIoU sobre $\mathcal{C}_n$ y construye sus prototipos sobre un descriptor geométrico analítico, con linealidad, planaridad y dispersión, que la literatura auto-supervisada considera un atajo perjudicial. La hipótesis de investigación es que reemplazarlo por una representación auto-supervisada confiable aumenta de forma medible el mIoU sobre las clases novedosas en un levantamiento patrimonial real.

Objetivos de investigación

Objetivo General Construir y medir un modelo híbrido que integre las representaciones auto-supervisadas de Sonata en el módulo de prototipos de HyperNCD, y cuantificar la mejora del mIoU en la clasificación de elementos estructurales no anotados sobre el levantamiento de la Catedral de Lima.

Objetivos Específicos El proyecto declara **dos objetivos con trabajo computacional propio**, OE1 y OE2, y **uno de salida analítica**, OE3.

OE1. Integrar Sonata en el módulo de prototipos. Es el objetivo central. HyperNCD describe cada punto con un vector F_i que junta f_{sem}, el descriptor semántico de su red supervisada, y f_{geo}, el geométrico calculado a mano (Ec. (II.9)). Se implementa el bloque de fusión que sustituye f_{geo} por f_{ssl}, el descriptor del codificador congelado de Sonata, y resuelve la diferencia de resolución entre PTv3 y la rejilla de Minkowski con *up-cast* y *cache* por sección.

- *Herramientas y trabajo computacional:* pesos públicos de Sonata, Pointcept, PTv3, `spconv`, MinkowskiEngine y PyTorch; módulo de fusión e inferencia congelada sobre cada sección.

- *Resultado verificable:* repositorio con el híbrido produciendo $F_i = [f_{\text{sem}}; f_{\text{ssl}}]$ y una **prueba de funcionamiento** sobre una sección de control donde el mIoU de las clases conocidas no baje respecto de la línea base.
- *Conclusión asociada:* si la sustitución es viable dentro del hipergrafo.

OE2. Calibrar la configuración del híbrido. Determinar α, que balancea la componente geométrica y la semántica, M, el número de vecinos por hiperarista, y ϵ, que suaviza las etiquetas, con los que se maximiza el mIoU de las clases novedosas en una sección de referencia. Al combinar dos métodos, las ventajas y los modos de falla de cada uno pasan al resultado.

- *Herramientas y trabajo computacional:* búsqueda por grilla acotada, AdamW y Weights & Biases; barrido sobre la sección de referencia y fijación de la configuración, aplicada sin reajuste al resto.
- *Resultado verificable:* curvas de sensibilidad de los tres parámetros y la configuración fijada, con la métrica reportada también en el punto intermedio del proceso (Sección 2.4.10).
- *Conclusión asociada:* qué parámetro domina y de qué etapa proviene la mejora.

OE3. Cuantificar la mejora y delimitar el rango de operación. No incorpora trabajo computacional nuevo. Es la salida de análisis de OE1 y OE2. Consiste en medir el ΔmIoU de las clases novedosas (Ec. (II.24)) frente a la línea base, identificar qué variable domina (Ec. (II.19)), comparar las tres variantes del descriptor , geométrica, auto-supervisada y concatenación, y fijar el rango favorable.

- *Herramientas:* el mIoU con emparejamiento húngaro, que decide qué grupo descubierto corresponde a qué clase real; los índices ARI y NMI; la brecha cerrada G (Ec. (II.25)); y visualizaciones PCA.
- *Resultado verificable:* tabla comparativa, estudio de ablación y curva de degradación que fija el rango declarado.

Justificación

La clasificación de elementos estructurales se hace hoy a mano porque no hay herramienta que cubra la tarea cuando la lista de clases no está cerrada, y cada punto

de mIoU ganado es corrección que el especialista deja de hacer. Los usuarios están conformes con la precisión que obtienen a mano, así que el valor está en automatizar la tarea a un nivel suficiente, no en igualar esa precisión.

El aporte no es una arquitectura nueva. Lo que se busca es establecer si un problema ya demostrado con supervisión completa se mantiene cuando no hay etiquetas que corrijan al modelo. Es un resultado incierto, medible y que ningún trabajo previo ha reportado, y como se mueve una sola pieza, cualquier variación se puede atribuir al reemplazo.

Alcance y limitaciones

Objeto de estudio y generalización Se trabaja sobre un único edificio, la **Catedral de Lima**, con el levantamiento provisto por la Facultad de Ingeniería Civil. El objeto es fijo y el modelo se ajusta a él, no al revés. Los resultados se declaran válidos para tipología de catedral o iglesia colonial, es decir nave con columnas y arcos, cubierta abovedada y fachada con torres, y no se extienden a vivienda ni a obra vial o industrial.

Volumen de datos El levantamiento se recibe como archivos LAS (Sección 2.1.3) y la unidad de trabajo es la **sección espacial** (Sección 2.1.9). Una sección es el conjunto de archivos que se preprocesan juntos, y las secciones funcionan como particiones de entrenamiento y evaluación. El preprocesamiento implementado aplica dos reducciones de naturaleza distinta (Tabla A). El filtrado SOR descarta los puntos estadísticamente aislados, que en el escaneo de prueba son el 4,73 %. La voxelización no descarta ruido. Agrupa los puntos que caen en un mismo cubo de 5 cm y los representa con uno solo, de modo que 8 184 419 puntos quedan en 610 637 representantes, el 7,46 % de los registros anteriores. El 92,54 % restante no era ruido, sino mediciones redundantes de superficies que el representante de su vóxel sigue

describiendo. El vóxel de 5 cm es un límite, no una preferencia. Por debajo, con muy pocos puntos por clase novedosa, la diferencia entre el modelo base y el híbrido dejaría de distinguirse del ruido entre corridas.

TABLA A: Cadena de reducción de datos medida sobre un escaneo del levantamiento (archivo `PL7_SCN0001.las`), con los conteos que registra el propio preprocesamiento.

Etapa	**Volumen**	**Recurso**
Archivo LAS en bruto	8 590 904 puntos, 292 MB	Almacenamiento
Filtrado SOR, $q = 20$ y $\lambda = 2$	8 184 419 puntos (95,27 %)	CPU
Voxelización a 5 cm	610 637 representantes (7,46 % de los anteriores)	CPU
Matriz de salida	$610\,637 \times 9$ en precisión simple, 22 MB	Almacenamiento
Sonata congelado, una pasada con *cache*	Una vez por sección	1 GPU

Hardware El proyecto está limitado por cómputo. Se ejecuta sobre el clúster **Khipu** de UTEC, cuya configuración de GPU cambió hace poco, de modo que **no se estiman tiempos de calendario**. Se especifica el hardware sobre el que las pruebas corrieron realmente (Tabla B). La división en secciones sostiene esta política, ya que ninguna etapa necesita la nube completa en memoria.

TABLA B: Requisitos del método y recursos disponibles en el clúster Khipu (UTEC), verificados por los autores sobre los nodos asignados.

Recurso	**Requisito del método**	**Disponible y verificado**
GPU	CUDA $\geq$ 8.0, una sola unidad; el diseño por secciones evita el multi-GPU	1× RTX A6000 (`ds001`, `g002`); A100 con particiones MIG en `ag001`
VRAM	Pico medido de 34,60 GB en PL13 (803 099 puntos); los pesos del codificador (108,46 M parámetros) ocupan 0,43 GB	48 GB (49 140 MiB)
RAM del nodo	Debe alojar la sección completa antes de voxelizar	64 GB solicitados por corrida
Almacenamiento	*Cache* de f_{ssl}, legible desde ambos entornos	`/home` compartido, 16 TB libres; `/local` por nodo
Toolkit CUDA	11.3 para MinkowskiEngine (HyperNCD) y 12.4 para PTv3 (Sonata)	11.8 y 12.4 vía módulos; driver 610.57.04
Conectividad	Descarga de pesos preentrenados y datos	Solo desde el nodo de acceso
Gestor de colas	Envío no interactivo de corridas largas	Slurm 25.11.5.1

El nodo `ag001` ofrece además una A100 con particiones MIG, útiles para pruebas cortas. Los nodos de cómputo no tienen salida a internet, así que los pesos y los datos se descargan desde el nodo de acceso.

Costo computacional medido El codificador de Sonata corre congelado y una sola vez por sección, así que su costo no se repite en cada época. En el escaneo PL13, con 796 315 puntos tras la rejilla de 2 cm, la inferencia tardó 13,9 segundos en la RTX A6000 y usó 34,60 GB de memoria de vídeo (Sección 3.5.2).

Rangos de operación El proyecto declara por adelantado dónde se espera que el método funcione, y OE3 lo verifica. Se espera que funcione con densidad de al menos 1 pt/cm^2, con elementos de 20 cm o más de firma geométrica repetida, como columnas, arcos, bóvedas, cornisas y contrafuertes, y hasta seis clases novedosas por sección. No se espera buen comportamiento con ornamento fino de menos de 5 cm, superficies muy ocluidas, ni clases novedosas bajo el 0,5 % de los puntos, régimen en el que HyperNCD colapsa a 11,8 mIoU [13], ni con muchas clases simultáneas, donde Sonata baja a 29,3 mIoU en ScanNet200 frente a 72,5 en ScanNet [7].

Exclusiones Quedan fuera el preentrenamiento auto-supervisado propio, que en Sonata exigió 140 mil escenas y 32 GPU, la detección de instancias, el vocabulario abierto y la generación del modelo BIM.

CAPÍTULO I

REVISIÓN CRÍTICA DE LA LITERATURA

Las cuatro publicaciones revisadas se corresponden con los objetivos específicos de la tesis según la Tabla 1.1. De ellas nacen los **siete aportes** que el proyecto declara. El número de aportes no coincide con el de publicaciones porque un mismo estudio puede dar origen a más de uno, según cuántos de sus resultados se estén explotando. De HyperNCD nacen dos, de BRDL tres, y de Sonata y de PTv3 uno cada uno.

Tabla 1.1: Correspondencia entre las publicaciones revisadas, lo que aportan y el objetivo específico con el que se identifican.

Publicación	**Qué aporta a la tesis**	**Aportes**	**Objetivo**
Zhang et al. (2026), HyperNCD (Sección 1.1)	El método sobre el que se construye y la vara de comparación	A1, A2	OE1
Wu et al. (2025), Sonata (Sección 1.2)	La pieza que sustituye al descriptor y la forma de medir representación	A3	OE1, OE3
Wu et al. (2024), PTv3 (Sección 1.3)	La estructura espacial que crea el problema de integración	A4	OE1
Su et al. (2025), BRDL (Sección 1.4)	El procedimiento de calibración bajo desbalance de clases	A5, A6, A7	OE2

1.1 Zhang, Wu, Li, Han y Shen (2026): HyperNCD, razonamiento por hipergrafo

Estudio relevante. Zhang, Wu, Li, Han y Shen (2026), *Geometric-Aware Hypergraph Reasoning for Novel Class Discovery in Point Cloud Segmentation*, publicado en CVPR 2026 [13]. Es el estado del arte del problema y el método sobre el que esta tesis construye. Se identifica con **OE1**.

Aporte al proyecto.

Qué hizo. Representa las relaciones entre clases con un **hipergrafo**. En un grafo común una arista une dos nodos; en un hipergrafo una hiperarista puede unir varios a la vez, lo que permite expresar que un conjunto de clases comparte una misma estructura geométrica. Los nodos son los prototipos de clase, y cada prototipo se construye sobre un descriptor compuesto: un descriptor semántico f_{sem} que produce la red supervisada y un descriptor geométrico f_{geo} calculado con una fórmula fija.

Qué resultado obtuvo. Sobre SemanticKITTI obtiene entre 11,8 y 47,5 de mIoU en clases novedosas según la partición, frente a 49,8 del equivalente supervisado; sobre SemanticPOSS, entre 22,3 y 47,2 frente a 50,6 [13, Tablas 1 y 2]. Más importante que el agregado es su estudio de ablación, que apaga una pieza a la vez para ver cuánto aporta cada una. Sobre la partición 0 de SemanticPOSS, partiendo de una red base que rinde 23,2 de mIoU, el prototipo geométrico aporta +4,2 puntos y el hipergrafo +9,6 sobre esa misma base (Tabla 1.2).

TABLA 1.2: Estudio de ablación de HyperNCD sobre la partición 0 de SemanticPOSS, según la Tabla 3 de [13]. Es la evidencia cuantitativa que sostiene la hipótesis de esta tesis.

Configuración	mIoU promedio	Ganancia sobre la base
Red base (MinkowskiUNet-34C modificada)	23,2	referencia
Solo prototipo consciente de la geometría	27,4	+4,2
Solo hipergrafo	32,8	+9,6
Prototipo geométrico e hipergrafo	34,2	+11,0
Método completo, con hiperaristas dinámicas	36,5	+13,3

Por qué sirve. La ablación deja al descubierto un desequilibrio que los autores no comentan. El descriptor geométrico aporta menos de la mitad que el mecanismo que lo consume. Esto no significa que f_{geo} sea inútil. Significa que aporta poco en relación con el lugar central que ocupa dentro del método, y por tanto que una representación aprendida puesta en su lugar debería aportar más.

La limitación de fondo es la naturaleza de ese descriptor. f_{geo} se calcula de forma analítica. Se toman los 15 vecinos más cercanos, se arma la matriz de covarianza de sus distancias, se obtienen sus autovalores y de ellos se derivan la linealidad, la planaridad y la dispersión [13]. Ese cálculo es siempre el mismo y no aprende nada del dominio en el que se aplica; es, además, el tipo de señal que la Sección 1.2 caracteriza como un atajo perjudicial. Una segunda limitación es la dispersión entre particiones, entre 11,8 y 47,5, que los propios autores atribuyen al desbalance de clases. Una tercera no está en el artículo sino en lo que omite, porque **no reporta ninguna cifra de cómputo**, ni GPU, ni memoria, ni tiempo.

De aquí el proyecto toma la arquitectura y su código público; los hiperparámetros publicados, que fijan el punto de partida de la calibración (15 vecinos, $M = 8$, $\alpha = 0{,}5$, $\epsilon = 0{,}15$); y la vara contra la cual se mide el aporte principal, porque cualquier reemplazo de f_{geo} debe compararse contra sus +4,2 puntos.

1.2 Wu et al. (2025): Sonata y el atajo geométrico

Estudio relevante. Wu, DeTone, Frost, Shen, Xie, Yang, Engel, Newcombe, Zhao y Straub (2025), *Sonata: Self-Supervised Learning of Reliable Point Representations*, publicado en CVPR 2025 [7]. Se identifica con **OE1** y, por su metodología de medición, con **OE3**.

Aporte al proyecto.

Qué hizo. Identifica y ataca el **atajo geométrico**. Cuando un modelo se entrena sin etiquetas sobre nubes de puntos, la representación tiende a colapsar sobre pistas geométricas de bajo nivel muy fáciles de leer, como la normal de la superficie o la altura del punto, en lugar de aprender semántica. Es un problema propio del 3D, porque la geometría entra por las coordenadas mismas y los operadores la usan

de forma directa. Sonata lo ataca oscureciendo la información espacial dentro de un esquema de auto-destilación entrenado sobre 140 mil escenas.

Qué resultado obtuvo. La evidencia se mide por *linear probing*, que consiste en que se congelan todos los pesos del codificador y se entrena encima únicamente un clasificador lineal, con menos del 0,2 % de los parámetros. Como el codificador no puede cambiar, lo que se mide es la calidad de la representación y nada más. Sobre ScanNet el desempeño pasa de 5,6 a 72,5 de mIoU (Tabla 1.3); en S3DIS Área 5 alcanza 72,3 con clasificador lineal, prácticamente a la par de una red supervisada completa, que obtiene 72,4.

TABLA 1.3: *Linear probing* sobre ScanNet, conjunto de validación, según la Tabla 5 de [7]. Todos los métodos congelan el codificador y entrenan una única capa lineal, así que la columna mide calidad de la representación y no capacidad del clasificador.

Método	mIoU (%)
PointContrast	5,6
CSC	12,6
MSC	21,8
DINOv2, agregado de 2D a 3D	63,1
Sonata	**72,5**

Por qué sirve. Por dos razones. La primera es el resultado. La señal geométrica de bajo nivel, medida en igualdad de condiciones, es un mal descriptor, y existe un codificador público, ya entrenado y congelable, que la reemplaza sin exigir preentrenamiento propio. Es la pieza que ocupa el lugar de f_{geo} en el aporte A1. La segunda es de método. El *linear probing* mide calidad de representación de forma aislada, congelando todo lo demás, y ese principio es el que permite separar después lo que aporta la representación de lo que aporta el razonamiento.

El trabajo declara tres limitaciones que acotan el riesgo del proyecto:

1. *Dominio de preentrenamiento.* El modelo se preentrena sobre escenas interiores densas, entre ellas ScanNet, S3DIS y Structured3D [7]. El levantamiento de una catedral cae del lado favorable de esa frontera, pero el cambio de dominio sigue siendo el principal riesgo declarado.

2. *Régimen congelado en exteriores.* En escaneo láser urbano el *linear probing* queda por debajo de una PTv3 supervisada, con 62,0 frente a 69,1 de mIoU en SemanticKITTI, aunque con ajuste fino completo Sonata alcanza 72,6 y la supera [7, Tabla 8]. La desventaja es del régimen congelado y no de la representación, y el régimen congelado es exactamente el que adopta este proyecto.
3. *Vocabularios grandes.* Con 200 clases la discriminación cae a 29,3 de mIoU en ScanNet200, frente a 72,5 con 20 clases en ScanNet. Esto fija el límite superior de clases novedosas simultáneas que el proyecto puede declarar como rango favorable.

1.3 Wu et al. (2024): Point Transformer V3 y el problema de integración

Estudio relevante. Wu, Jiang, Wang, Liu, Liu, Qiao, Ouyang, He y Zhao (2024), *Point Transformer V3: Simpler, Faster, Stronger*, publicado en CVPR 2024 [6]. Es la red sobre la que está construido Sonata, y el codificador que este proyecto emplea congelado es una PTv3 con pesos auto-supervisados. Se identifica con **OE1**.

Aporte al proyecto.

Qué hizo. Sustituyó la vecindad calculada por vecinos más cercanos por una vecindad *serializada*. Los puntos se cuantizan a una rejilla entera, se ordenan siguiendo una curva de llenado de espacio, que recorre el volumen visitando cada celda una sola vez y manteniendo juntos los puntos cercanos, y la secuencia resultante se corta en parches contiguos dentro de los cuales se calcula la atención. El trabajo emplea cuatro recorridos, orden Z, orden de Hilbert y sus dos variantes transpuestas, y alterna entre ellos.

Qué resultado obtuvo. La serialización amplía el campo receptivo de 16 a 1024 puntos. La consulta de vecinos más cercanos y la codificación posicional relativa ocupaban el 54 % del tiempo de paso hacia adelante de la versión anterior, y PTv3 elimina ambas. Su tabla de eficiencia, medida sobre nuScenes con lote 1 en una sola RTX 4090, da los valores de la Tabla 1.4.

Tabla 1.4: Eficiencia de PTv3 frente a la red dispersa con la que tendrá que convivir en el modelo híbrido, según la Tabla 1 de [6]. Medido sobre nuScenes, lote de tamaño 1, una sola RTX 4090.

		Entrenamiento		Inferencia	
Método	Parám.	Latencia	Memoria	Latencia	Memoria
MinkUNet, núcleo 3	37,9 M	163 ms	3,3 G	48 ms	1,7 G
PTv2, parche 16	12,8 M	213 ms	10,3 G	146 ms	12,3 G
PTv3, parche 1024	46,2 M	**119 ms**	**3,3 G**	**44 ms**	**1,2 G**

Por qué sirve. Porque aquí aparece el problema que este proyecto debe resolver y que ninguno de los dos estudios anteriores plantea, ya que ninguno intenta juntar las piezas. Las dos ramas del híbrido parten de las mismas coordenadas pero definen la vecindad de maneras distintas. HyperNCD opera sobre una **rejilla dispersa de Minkowski**, donde el vecino de un punto son los que caen dentro del núcleo de la convolución, o sea, adyacencia métrica en el volumen. Sonata, a través de PTv3, opera sobre una **secuencia unidimensional**, donde el vecino es el que quedó al lado dentro de un parche de 1024 elementos de la curva. Una curva de llenado de espacio preserva la localidad de forma aproximada y no exacta. Dos puntos contiguos en el volumen pueden quedar separados en la secuencia si caen a ambos lados de un giro. Pasar la salida de una rama a la otra exige absorber ese desajuste de forma explícita.

Se suma una segunda incompatibilidad, de ingeniería y no de arquitectura, que la inspección del código de ambos grupos deja en evidencia que las dos ramas exigen **pilas de dependencias mutuamente excluyentes**, MinkowskiEngine contra CUDA 11.3 (en Khipu se compiló con 11.8) y PTv3 contra CUDA 12.4. Integrar las dos implementaciones de referencia en un único proceso no es posible sin renunciar a una de las dos, de modo que el punto de fusión no puede ser un objeto en memoria compartido.

La Tabla 1.4 muestra además que PTv3 no es la pieza cara del sistema,

porque con 46,2 M de parámetros consume menos memoria de inferencia que MinkUNet, que tiene 37,9 M, y es más rápido. El costo del híbrido no viene de añadir un transformador, sino de ejecutarlo dentro del bucle de entrenamiento. Y lo que hace resoluble todo lo anterior es que el codificador se emplea congelado y solo en inferencia. Su salida no depende de la época ni del estado del optimizador, así que puede calcularse una sola vez por sección.

1.4 Su, Chen y Wang (2025): BRDL, calibración bajo desbalance de clases

Estudio relevante. Su, Chen y Wang (2025), *Balanced Residual Distillation Learning for 3D Point Cloud Class-Incremental Semantic Segmentation* [10]. Se identifica con **OE2**.

Aporte al proyecto.

Qué hizo. Aborda la pregunta que plantea OE2, es decir, cómo se ajusta un sistema que debe incorporar clases sobre las que no tuvo supervisión completa sin degradar las que ya clasificaba bien. Combina dos piezas, una destilación residual que refina el conocimiento previo ampliando la red y un aprendizaje balanceado de pseudo-etiquetas que corrige el sesgo hacia las clases base. El escenario no es el mismo que el de esta tesis. BRDL trabaja en aprendizaje incremental por clases, donde las categorías nuevas sí llegan etiquetadas cuando les toca su turno, mientras que en descubrimiento de clases novedosas nunca lo están.

Qué resultado obtuvo. Interesan tres bloques. Primero, el barrido de un solo hiperparámetro. Al mover λ_2 entre 0,1 y 1,0 el mIoU recorre el intervalo de 45,1 a 46,28, con el máximo en $\lambda_2 = 0{,}2$ [10, Tabla 3]. Segundo, la grilla conjunta de dos hiperparámetros acoplados. Al barrer λ_2 y λ_3 a la vez el máximo sube a 46,58 en la combinación (0,2; 0,5) y el mínimo se mantiene en 45,1, con lo que toda la grilla

cabe en 1,48 puntos de mIoU [10, Tabla 4]. Tercero, la ablación de las dos estrategias (Tabla 1.5). A ello se suman dos datos que el artículo publica sin comentar. En el entrenamiento multi-paso una clase se queda en 0,00 de mIoU en los cinco pasos y para los dos métodos comparados, y otra cae de 34,42 a 0,07 [10, Tabla 5]; y el costo incremental de la modificación se reporta de forma explícita, con 142 a 144 segundos por época y 1608 a 1614 KB de memoria [10, Tabla 8].

TABLA 1.5: Ablación de las dos estrategias de BRDL sobre S3DIS, partición S0, según la Tabla 6 de [10]. La fila intermedia es la que importa para esta tesis, y es que una de las dos estrategias, aplicada por separado, deja el resultado por debajo de no aplicar ninguna.

Configuración	mIoU (*all*)	Diferencia
Sin ninguna de las dos estrategias	45,19	referencia
Solo una de las dos estrategias	44,90	−0,29
Solo la otra estrategia	46,83	+1,64
Las dos estrategias juntas	47,26	+2,07

Por qué sirve. Por tres razones. La primera es metodológica. Es el único trabajo revisado que publica una grilla conjunta de dos parámetros acoplados y la lee como tal, concluyendo que λ_2 domina y que λ_3 solo introduce ajustes finos que amplifican o amortiguan el efecto según el valor de λ_2. El efecto no es separable, y una calibración parámetro por parámetro llega a un óptimo distinto del de la grilla conjunta. La Tabla 1.5 refuerza el punto desde el lado de los componentes. Activar una sola de las dos piezas empeora el resultado respecto de no activar ninguna, y solo la combinación gana.

La segunda es de magnitud. Toda la grilla mueve 1,48 puntos de mIoU mientras que activar o desactivar los componentes mueve 2,07. La calibración es necesaria pero no es donde está la ganancia grande, y esa cifra fija la expectativa realista de OE2.

La tercera tiene que ver con el desbalance. El fracaso por falta de soporte es total y no gradual, con una clase en cero en los cinco pasos y para los dos métodos.

Ningún valor de hiperparámetro rescata una clase cuyo soporte es insuficiente. Eso coincide con el régimen que esta tesis declara desfavorable, el de clases novedosas por debajo del 0,5 % de los puntos de la sección, y explica por qué HyperNCD necesita a la vez el ajuste dinámico de hiperaristas y el suavizado de etiquetas ϵ. Las cifras de BRDL no se pueden trasladar por la diferencia de escenario; lo que se traslada es el procedimiento y el diagnóstico.

1.5 Conclusión de la revisión crítica

Los estudios previos resuelven el problema **solo de forma parcial**. Existe un método de razonamiento entre clases que funciona, pero su descriptor geométrico aporta menos de la mitad que el mecanismo que lo consume (Sección 1.1). Existe una representación de puntos de calidad demostrada, pero medida en régimen supervisado y de vocabulario cerrado (Sección 1.2). Existe una caracterización precisa de la estructura espacial sobre la que esa representación opera, y por eso mismo se sabe que no coincide con la del método anterior (Sección 1.3). Existe un procedimiento publicado para calibrar mecanismos acoplados, pero sobre parámetros de un mismo método y de una misma escala (Sección 1.4). Ninguno de esos trabajos se ha evaluado sobre patrimonio construido, y nadie ha juntado las piezas.

Sobre esa base el proyecto declara **siete aportes**. Se enuncian a continuación, agrupados por objetivo específico, y la Tabla 1.6 los reúne.

OE1. Integración de Sonata en el módulo de prototipos. Tres aportes. Sustituir el descriptor geométrico analítico por el codificador auto-supervisado congelado (A1), corregir la base de comparación a partir de la auditoría del código liberado (A2) y resolver el puente entre las dos estructuras espaciales que hace viable esa sustitución (A4).

A1 Sustitución del descriptor geométrico (OE1). Reemplazar f_{geo} por f_{ssl}, el codificador congelado de Sonata, conservando íntegro el razonamiento por hipergrafo. Así, $F_i = [f_{\text{sem}};\ f_{\text{ssl}}]$ y cualquier variación de la métrica es atribuible a ese reemplazo y a nada más. *Resultado verificable:* el híbrido corriendo de extremo a extremo sobre una sección de control, con las dimensiones de ambas ramas documentadas y sin que el mIoU de las clases conocidas baje respecto de la línea base.

A2 Auditoría de la implementación de referencia y corrección de la línea base (OE1). La inspección del código liberado muestra que el módulo que reproduce la formulación publicada del razonamiento por hipergrafo no se invoca desde ninguna ruta de entrenamiento ni de evaluación; que las matrices de peso de sus capas se instancian dentro del cuerpo de la función en cada llamada, por lo que no se registran entre los parámetros del modelo, no reciben gradiente y se reinicializan al azar en cada invocación; que el procedimiento llamado de actualización dinámica reproduce término por término el de construcción inicial sobre la misma matriz de similitud y, en la práctica, no actualiza nada; y que el descriptor geométrico ejecutado tiene dimensión seis, porque incorpora el centroide de la región, y no tres como describe el artículo. Esto obliga a tres decisiones. La línea base se establece sobre el código que se ejecuta y no sobre la formulación publicada, la comparación se hace contra corridas propias bajo condiciones idénticas de datos, partición y semilla, y la sustitución se aplica sobre el descriptor de dimensión seis. Sin esta corrección, cualquier mejora reportada quedaría mal atribuida. *Resultado verificable:* la tabla de correspondencia entre artículo e implementación y las tres decisiones metodológicas que de ella se derivan.

A4 Puente entre dos estructuras espaciales incompatibles (OE1). Resolver el emparejamiento entre la secuencia serializada de PTv3 y la rejilla dispersa de Minkowski mediante *up-cast* de resolución y *cache* de características por sección. La misma decisión resuelve tres problemas a la vez: el emparejamiento de vecindades, la incompatibilidad entre pilas de dependencias, que deja de importar porque las dos ramas nunca coexisten en un mismo proceso, y la repetición del costo del codificador en cada época, que desaparece. Incluye la formulación propia de una cota de aceleración que delimita hasta dónde vale la pena distribuir el cacheo. *Resultado verificable:* el *cache* persistente por sección, legible desde los dos entornos, y la cota de aceleración. El *cache* ya se generó para el escaneo PL13 (Sección 3.5.2); falta que la rama de HyperNCD lo lea.

OE2. Calibración del híbrido. Tres aportes. Hacer comparables dos ramas de escalas distintas (A5), calibrar de forma conjunta los tres parámetros que gobiernan

la afinidad entre regiones (A6) y verificar que la configuración elegida se sostiene fuera de la sección donde se calibró (A7).

A5 Normalización de escala entre dos ramas heterogéneas (OE2). El descriptor híbrido $F_i = [f_{\text{sem}}; f_{\text{ssl}}]$ une dos ramas de escala distinta, con 96 componentes entrenadas junto con el resto del modelo y 1088 componentes congeladas cuyo rango no está normalizado contra el de la rama semántica. Sin corrección, la rama más ancha domina la similitud por coseno S_s con la que se comparan las regiones, y el valor publicado $\alpha = 0{,}5$ deja de repartir el peso entre forma y significado como en la línea base. Hacer comparables las dos ramas antes de concatenarlas es una requisito previo a la calibración, y el régimen congelado lo agrava porque impide que la rama auto-supervisada se reescale sola durante el entrenamiento. En BRDL los dos parámetros pertenecen al mismo método y comparten escala, por lo que este caso queda fuera de lo que ese trabajo cubre. *Resultado verificable:* el procedimiento de normalización adoptado y su efecto medido sobre la distribución de S_b antes y después.

A6 Calibración conjunta y verificación de no-aditividad (OE2). Determinar los tres valores con una grilla conjunta y curvas de sensibilidad sobre una sección de referencia, en lugar de un barrido por parámetro, y verificar si la no-aditividad que BRDL documenta entre sus dos estrategias aparece también entre la representación y el razonamiento del híbrido. *Resultado verificable:* la grilla conjunta, las tres curvas de sensibilidad y la configuración fijada.

A7 Verificación de que la configuración se sostiene fuera de donde se calibró (OE2). La configuración se fija sobre una única sección de referencia y se aplica **sin reajuste** a las restantes, reportando la degradación observada. Así se distingue una calibración real de un ajuste hecho sobre aquello que se iba a medir, y el estudio de referencia muestra por qué importa. Al cambiar solo el orden de presentación de las clases, los métodos incrementales existentes se degradan de forma significativa mientras que el entrenamiento conjunto apenas se mueve. *Resultado verificable:* la métrica por sección bajo configuración fija y su diferencia respecto de la sección de calibración.

OE3. Cuantificación de la mejora y delimitación del rango de operación. Un aporte. Formular una métrica que separe la contribución de la representación de la del razonamiento, junto con el protocolo de atribución por etapas que la acompaña (A3). El estudio de Sonata (Sección 1.2) sostiene, junto con la Sección 1.1, el aporte A1, porque aporta la pieza que se coloca en lugar de f_{geo} y la evidencia de que hacerlo tiene sentido. Verificar si la nocividad del atajo geométrico, demostrada en

régimen supervisado, se mantiene en régimen de descubrimiento de clases novedosas es la pregunta que A1 responde. De su metodología nace además un aporte propio:

A3 Métrica de brecha cerrada G y protocolo de atribución por etapas (OE3). Las métricas estándar no separan lo que aporta la representación de lo que aporta el razonamiento. El mIoU final es el mismo número venga de donde venga la ganancia. Trasladando el principio del *linear probing*, este trabajo formula la brecha cerrada G (Ec. (II.25)), que expresa qué parte de la distancia entre clases conocidas y novedosas se logró recuperar, y mide también en el punto intermedio del proceso (Sección 2.4.10). Incorpora además un análisis de dominancia que identifica qué clase gobierna cada mejora agregada, para que un promedio favorable no oculte una única clase que arrastra al resto. *Resultado verificable:* G formulada, la medición intermedia y las proyecciones PCA de los descriptores.

Tabla 1.6: Inventario de los siete aportes del proyecto. Cada uno nace de un estudio revisado y termina en un resultado propio que se puede verificar.

	Aporte	**Nace de**	**Resultado verificable**
OE1. Integración de Sonata en el módulo de prototipos			
A1	Sustitución de f_{geo} por f_{ssl}	HyperNCD (§1.1), Sonata (§1.2)	Híbrido de extremo a extremo sin caída en clases conocidas
A2	Auditoría del código y corrección de la línea base	HyperNCD (§1.1) y su repositorio	Tabla artículo frente a implementación y tres decisiones metodológicas
A4	Puente entre serialización y rejilla dispersa	PTv3 (§1.3)	*Cache* por sección y cota de aceleración propia
OE2. Calibración del híbrido			
A5	Normalización de escala entre ramas heterogéneas	BRDL (§1.4)	Distribución de S_b antes y después
A6	Calibración conjunta de (α, M, ϵ) y no-aditividad	BRDL (§1.4)	Grilla conjunta, curvas de sensibilidad, configuración fijada
A7	Robustez de la configuración entre secciones	BRDL (§1.4)	Métrica por sección sin reajuste
OE3. Cuantificación de la mejora y rango de operación			
A3	Brecha cerrada G y atribución por etapas	Sonata (§1.2)	G formulada, medición intermedia, PCA

El aporte al estado del arte no es una arquitectura nueva. Es establecer si un hallazgo demostrado en un régimen se transfiere a otro donde resulta más crítico,

hacerlo sobre un objeto de estudio en el que el costo del trabajo manual es real y medible, y dejar documentado con qué base de comparación y a qué costo se hizo.

CAPÍTULO II

MARCO TEÓRICO

Este capítulo define los conceptos que emplea el trabajo, en cinco secciones: los de ingeniería civil sobre el objeto de estudio (Sección 2.1), los de computación (Sección 2.2), los algoritmos (Sección 2.3), las métricas de evaluación (Sección 2.4) y el diseño de la tesis, que los integra (Sección 2.5).

Solo se definen conceptos que el diseño usa, y cada componente del diagrama de diseño (Figura 2.3) enlaza con la definición que le corresponde. Las fórmulas se explican aquí. Cada símbolo de una fórmula, en azul, enlaza con su significado en la nomenclatura del Anexo B.

2.1 Ingeniería Civil y Conservación del Patrimonio

La ingeniería civil es clave para la conservación del patrimonio arquitectónico, ya que proporciona las herramientas necesarias para la documentación, el análisis y la preservación de estructuras históricas como la Catedral de Lima. A continuación se presentan los conceptos fundamentales de esta disciplina que son relevantes para este trabajo, desde la nube de puntos y el levantamiento que la produce hasta su preparación y su segmentación.

2.1.1 Nube de Puntos

Una nube de puntos es un conjunto $P = \{(x_i, f_i)\}_{i=1}^{N}$ donde $x_i \in \mathbb{R}^3$ es la posición medida de un punto de la superficie y f_i sus atributos. No tiene conectividad

ni orden. Es una lista de mediciones, no una malla. Se almacena como una tabla con una fila por punto y una columna por valor. En este trabajo los atributos son el color, que registra el sensor, y la normal, que se estima a partir de los vecinos (Sección 2.1.8). Es el dato de entrada y el objeto que se segmenta.

2.1.2 Levantamiento Láser Terrestre (TLS)

El levantamiento láser terrestre es el registro de una edificación mediante un escáner láser estacionado en posiciones fijas. El escáner emite un haz, mide la distancia y la dirección de cada retorno y con ellas calcula la posición del punto en que el haz tocó una superficie. Es la técnica con la que se levantó la Catedral de Lima y, por tanto, la fuente de datos de este trabajo.

2.1.3 Formato LAS

El LAS es el formato binario estándar de la ASPRS para intercambiar nubes de puntos láser. Es el formato en que se recibe el levantamiento en bruto. Un archivo contiene una cabecera y un registro por punto. Cada registro guarda las coordenadas como enteros y atributos como la intensidad del retorno, la clase y, según el formato de registro, el color RGB en 16 bits. La cabecera declara el número de puntos y, por cada eje, una *escala* y un *desplazamiento* con los que se decodifican las coordenadas:

$$x = X \cdot s_x + o_x, \tag{II.1}$$

Con ella se recuperan las coordenadas reales, en metros, a partir de los enteros almacenados. Para y y z es análogo. En el archivo inspeccionado la escala es de 10^{-4} m. La normal no forma parte del formato, y en ese archivo el campo de clase vale cero en todos los puntos, es decir, el dato no trae etiquetas.

Como todos los registros tienen el mismo tamaño, el archivo puede leerse por bloques consecutivos. La implementación lee bloques de cinco millones de registros,

extrae de cada uno las coordenadas ya decodificadas y el color, y los apila en dos matrices de tres columnas. Los bloques se concatenan después en memoria, de modo que la lectura por bloques acota el tamaño de cada lectura pero la sección completa debe caber en la memoria del nodo.

De cada punto se toman seis valores, (x, y, z) y (R, G, B). El color se lleva de 16 bits al rango 0 a 255 dividiéndolo entre 256. El resto de campos no se usa.

2.1.4 Centrado de Coordenadas

El centrado es una traslación de la nube que lleva su centro al origen. Se calcula el centro de la caja que envuelve a la sección y se resta a todos los puntos:

$$c = \tfrac{1}{2}\left(\min{}_i x_i + \max{}_i x_i\right), \qquad x'_i = x_i - c, \tag{II.2}$$

con el mínimo y el máximo tomados eje por eje. El vector c se guarda junto a la salida para poder devolver cualquier resultado a su posición real sumándolo.

Las coordenadas del levantamiento son proyectadas, en metros, del orden de 10^5 en un eje horizontal y de 10^6 en el otro. La salida se guarda en precisión simple, que en el eje de mayor magnitud solo distingue pasos de un metro. Tras el centrado las coordenadas quedan en decenas de metros y conservan la resolución del escáner.

Este desplazamiento no es el de la cabecera LAS (Ec. (II.1)). El de la cabecera pertenece al formato y sirve para decodificar los enteros almacenados; el del centrado lo calcula el preprocesamiento sobre las coordenadas ya decodificadas. Sus valores tampoco coinciden. En el archivo inspeccionado el desplazamiento de cabecera en x es 278 802,58 m y el centro restado es 278 800,97 m.

2.1.5 Vecindad de los q Vecinos Más Cercanos

La vecindad de un punto i es el conjunto $\mathcal{N}_q(i)$ de los q puntos de la nube más próximos a él en distancia euclídea. Se abrevia KNN. La nube se indexa en un árbol k-d, que divide el espacio de forma recursiva y permite hallar los vecinos de un punto sin compararlo con todos los demás.

Es la noción de entorno local de un punto. Con ella se decide si un punto está aislado (Sección 2.1.6) y hacia dónde mira la superficie (Sección 2.1.8). En ambos casos $q = 20$.

2.1.6 Filtrado Estadístico de Valores Atípicos (SOR)

El filtrado estadístico de valores atípicos (SOR, *Statistical Outlier Removal*) descarta los puntos estadísticamente aislados. Un levantamiento en bruto contiene puntos que no pertenecen a ninguna superficie, como reflejos en vidrios, personas que cruzaron durante el escaneo y mediciones sueltas en el límite del alcance del sensor. Se reconocen porque quedan lejos de sus vecinos. Para cada punto se calcula la distancia media a sus q vecinos más cercanos (Sección 2.1.5),

$$\bar{d}_i = \frac{1}{q} \sum_{j \in \mathcal{N}_q(i)} \|x_i - x_j\|, \tag{II.3}$$

y el punto se conserva solo si

$$\bar{d}_i \leq \mu + \lambda\, \sigma, \tag{II.4}$$

Así se descartan los puntos que quedan lejos de sus vecinos. Un λ menor descarta más puntos, pero puede borrar detalle fino; un q mayor hace la estimación más estable.

Limpia la nube antes de voxelizarla, para que los puntos espurios no se promedien con los válidos. Con $q = 20$ y $\lambda = 2$, en el escaneo de prueba descarta 406 485 de 8 590 904 puntos, el 4,73 %.

2.1.7 Vóxel y Voxelización

Un vóxel es una celda cúbica de una rejilla regular que divide el espacio; su lado es el tamaño de vóxel. La voxelización es la discretización de la nube sobre esa rejilla. Los puntos que caen en un mismo vóxel v se agrupan y el grupo se sustituye por un único representante, cuya posición y color son el promedio de los del grupo:

$$\bar{x}_v = \frac{1}{|v|} \sum_{i \in v} x_i, \tag{II.5}$$

Con ello cada vóxel ocupado queda descrito por un solo punto. El representante no es, en general, ninguno de los puntos originales.

Reduce el número de registros y homogeniza la densidad, que en un levantamiento láser es muy alta cerca del escáner y baja lejos de él. No es un paso de limpieza. El SOR descarta puntos que no pertenecen a ninguna superficie; la voxelización conserva todas las superficies y solo cambia cuántos registros las describen. Con vóxeles de 5 cm, los 8 184 419 puntos del escaneo de prueba quedan en 610 637 representantes, el 7,46 % de los registros anteriores. El 92,54 % restante no era ruido. Eran mediciones redundantes de una misma porción de superficie.

Los Módulos 2 y 3 aplican además su propia discretización sobre esta salida (Secciones 2.2.6 y 2.2.9).

2.1.8 Normal de Superficie y su Estimación

La normal de un punto es el vector unitario $n_i = (n_x, n_y, n_z)$ perpendicular a la superficie en ese punto. Describe la orientación geométrica local, es decir, hacia dónde mira la superficie. No es una clasificación, porque no dice qué elemento es el punto. El escáner no mide la normal, así que se estima a partir de la vecindad del punto, formada por sus q vecinos más cercanos $p_1, \ldots, p_q$ (Sección 2.1.5). Primero

se calcula el centroide local,

$$\bar{p} = \frac{1}{q} \sum_{j=1}^{q} p_j. \tag{II.6}$$

El centroide local es el promedio de las posiciones de los vecinos cercanos y se utiliza como punto de referencia para estudiar cómo están distribuidos alrededor de la superficie. Después se calcula la matriz de covarianza,

$$C = \frac{1}{q} \sum_{j=1}^{q} (p_j - \bar{p}) (p_j - \bar{p})^\top . \tag{II.7}$$

La matriz de covarianza describe cómo se distribuyen los puntos alrededor del centroide y en qué direcciones presentan mayor o menor variación. Es una matriz simétrica de 3×3, y sus tres autovectores son esas direcciones principales, cada una con un autovalor que mide cuánto se dispersan los vecinos a lo largo de ella. Sobre una superficie, dos direcciones corresponden a su extensión, porque los vecinos se reparten a lo largo y a lo ancho, y la tercera, la de menor variación, corresponde aproximadamente a la perpendicular. La normal es por tanto el autovector de C asociado al menor autovalor.

El autovector fija una dirección pero no un sentido, ya que n y $-n$ son igual de válidos. La implementación resuelve la ambigüedad invirtiendo las normales que apuntan hacia abajo, de modo que todas cumplen $n_z \geq 0$. Esto las hace consistentes entre sí, pero no distingue la cara interior de un muro de la exterior.

Tres casos sirven de referencia. En una superficie horizontal, como un piso, la normal es aproximadamente vertical, $(0, 0, 1)$. En una pared vertical la normal es aproximadamente horizontal, con n_z cercano a cero. En una superficie inclinada, como el arranque de una bóveda, la normal tiene componentes en más de un eje.

Aporta tres de los nueve valores con que se describe cada punto a la entrada del codificador de Sonata. Se estima sobre los representantes que deja la voxelización, con $q = 20$, y no sobre los puntos en bruto.

2.1.9 Sección Espacial

Una sección es la porción del levantamiento que se preprocesa en una misma corrida. El preprocesamiento recibe uno o varios archivos LAS, los une y los trata como una sola sección; con un archivo por corrida, cada escaneo es una sección. Su salida se denota S y es una matriz de N filas, una por punto, y nueve columnas en precisión simple,

$$S = [\, x \;\; y \;\; z \;\; R \;\; G \;\; B \;\; n_x \;\; n_y \;\; n_z \,], \tag{II.8}$$

con la posición centrada en metros, el color entre 0 y 255 y la normal unitaria. Para el escaneo de prueba, $N = 610\,637$.

Es la unidad indivisible de trabajo, ya que ninguna etapa requiere la nube completa en memoria y ninguna sección depende de otra.

2.1.10 Segmentación Semántica

La segmentación semántica asigna una etiqueta de clase a cada punto de P. Se distingue de la segmentación de instancias, que además separa objetos distintos de una misma clase y que queda fuera del alcance de este trabajo.

2.1.11 Clase Conocida y Clase Novedosa

$\mathcal{C}_k$ es el subconjunto de clases con puntos etiquetados durante el entrenamiento y $\mathcal{C}_n$ el de clases presentes en la nube pero sin ninguna etiqueta, con $\mathcal{C}_k \cap \mathcal{C}_n = \emptyset$ y $\mathcal{C} = \mathcal{C}_k \cup \mathcal{C}_n$. En patrimonio, $\mathcal{C}_n$ recoge los elementos singulares de cada edificio, que no figuran en ningún catálogo previo.

2.2 Computación

Esta sección reúne las representaciones y estructuras de datos con que se procesa la nube de puntos.

2.2.1 Descriptor de Punto

Un descriptor es el vector numérico asociado a un punto o a una región que resume su contenido. HyperNCD [13] describe cada punto concatenando dos componentes:

$$F_i = \left[f_{\text{sem}}(x^i);\ f_{\text{geo}}(x^i) \right] . \tag{II.9}$$

Este trabajo sustituye el segundo componente por el descriptor auto-supervisado:

$$F_i = \left[f_{\text{sem}}(x^i);\ f_{\text{ssl}}(x^i) \right] . \tag{II.10}$$

Así cada punto se describe a la vez con lo aprendido de las clases conocidas y con la representación de Sonata. El punto y coma indica concatenación, es decir, colocar un vector a continuación del otro. La línea base describe así cada punto con $96+6 = 102$ números, y el modelo propuesto, con $96 + 1088 = 1184$.

2.2.1.1. Descriptor Semántico

Vector que la red troncal supervisada de HyperNCD entrega por cada punto. Esa red es una MinkowskiUNet-34C de 96 canales (Sección 2.2.8). Codifica a qué se parece un punto en términos de las clases que la red vio etiquetadas. No son medidas físicas, son coordenadas aprendidas, próximas entre puntos de una misma clase. Como la red se entrena, su salida cambia en cada época y debe recalcularse. Se denota f_{sem}.

2.2.1.2. Descriptor Geométrico Analítico

Vector calculado con una fórmula cerrada, sin aprendizaje, a partir de los autovalores $\lambda_1 \geq \lambda_2 \geq \lambda_3$ de la matriz de covarianza de un conjunto de puntos: linealidad $(\lambda_1 - \lambda_2)/\lambda_1$, planaridad $(\lambda_2 - \lambda_3)/\lambda_1$ y dispersión λ_3/λ_1. El artículo los calcula sobre un vecindario de 15 puntos; la implementación ejecutada los calcula sobre la región que contiene al punto (Sección 2.3.2) y les concatena el centroide de esa región, lo que da una dimensión total de 6. Este trabajo adopta la versión ejecutada, en la que cada punto hereda el descriptor de su región. Se denota f_{geo} y es el componente del descriptor que este trabajo sustituye. Solo describe forma, sin ningún conocimiento de qué es el punto.

2.2.1.3. Descriptor Auto-supervisado

Salida del codificador PTv3 preentrenado de Sonata [7], usado con pesos congelados. Su dimensión es 1088, resultado de concatenar las anchuras $512 + 384 + 192$ de las tres últimas etapas durante la propagación inversa del submuestreo. Se denota f_{ssl} y codifica estructura aprendida sobre 140 mil escenas, sin haber visto las clases de este proyecto. A diferencia de f_{geo}, no es un descriptor de forma. El preentrenamiento de Sonata está diseñado para evitar el atajo geométrico (Sección 2.2.4), de modo que f_{ssl} aporta al descriptor un conocimiento semántico adquirido sin etiquetas, complementario al supervisado de f_{sem}. Por eso el modelo es híbrido. Cada punto se describe a la vez con lo aprendido de las clases conocidas del proyecto y con lo aprendido de forma auto-supervisada fuera de él.

2.2.2 Aprendizaje Auto-supervisado

Régimen de entrenamiento en el que la señal de supervisión se construye a partir de los propios datos, por ejemplo vistas distintas de una misma escena que deben producir representaciones parecidas, y no de etiquetas humanas. Es el régimen con el que se preentrenó el codificador de Sonata; este trabajo no lo ejecuta, solo usa su resultado.

2.2.3 Codificador Congelado

Red cuyos pesos no se actualizan y que se usa exclusivamente en inferencia. Su costo se paga una sola vez y no en cada época, lo que permite calcular su salida por adelantado y almacenarla en disco. El codificador de Sonata tiene 108,46 M de parámetros, es decir, de pesos aprendidos, y su inferencia sobre PL13 usó 34,60 GB de memoria de vídeo (Sección 3.5.2).

2.2.4 Atajo Geométrico

Modo de falla del preentrenamiento auto-supervisado en tres dimensiones, diagnosticado por Sonata, en el que la representación colapsa sobre pistas geométricas fácilmente accesibles, como la normal, la altura o la curvatura local, en lugar de aprender semántica. Es la razón por la que este trabajo sustituye f_{geo}, y por la que el codificador de Sonata prescinde del decodificador.

2.2.5 Serialización

Ordenamiento de los puntos mediante una curva de llenado de espacio, de tipo Z o Hilbert, para que la proximidad en $\mathbb{R}^3$ se traduzca en proximidad dentro de

una secuencia unidimensional. La secuencia se corta en parches contiguos y dentro de cada uno se aplica atención, la operación con que un transformador combina la información de los elementos de un conjunto. Es el mecanismo que permite a PTv3 [6], la red del codificador, aplicar atención con costo lineal.

2.2.6 Rejilla Dispersa y Tensor Disperso

Una rejilla dispersa es una rejilla de vóxeles de la que solo se almacenan los ocupados. Un edificio ocupa una fracción mínima del volumen que lo envuelve, así que guardar también los vóxeles vacíos sería inviable. Un tensor disperso es la estructura de datos que la representa en MinkowskiEngine: una lista de coordenadas enteras de vóxel y, por cada una, un vector de características. Las coordenadas de los puntos se dividen entre el tamaño de vóxel, que es 5 cm por defecto, y se redondean a enteros. Si varios puntos caen en el mismo vóxel se conserva uno. En HyperNCD la característica inicial de cada vóxel ocupado es el valor constante 1, de modo que la red recibe únicamente la ocupación, es decir, la forma, y no el color ni la normal. Es la entrada de la red que produce f_{sem}.

2.2.7 Convolución Dispersa

La convolución dispersa es la operación de convolución definida sobre un tensor disperso. Como una convolución ordinaria, combina las características de un vóxel con las de sus vecinos dentro de un núcleo cúbico mediante pesos aprendidos, pero solo se evalúa en los vóxeles ocupados y solo considera vecinos ocupados. La vecindad es por tanto métrica: son vecinos los vóxeles adyacentes en el volumen. Es la capa con que está construida la red que produce f_{sem}, y su noción de vecindad es la que no coincide con la de la serialización (Sección 2.2.5).

2.2.8 MinkowskiUNet-34C

La MinkowskiUNet-34C es la red troncal de HyperNCD, una red de convoluciones dispersas con arquitectura codificador-decodificador. Tiene tres partes.

- **Codificador** (*encoder*). Cuatro niveles que reducen la resolución a la mitad cada vez y aumentan el número de canales, con anchuras $(32, 64, 128, 256)$. Cada nivel ve una porción mayor del edificio con menos detalle.
- **Decodificador** (*decoder*). Cuatro niveles que devuelven la resolución original, con anchuras $(256, 128, 96, 96)$.
- **Conexiones de salto** (*skip connections*). Cada nivel del decodificador recibe, concatenada, la salida del nivel del codificador que tiene su misma resolución. Así recupera el detalle fino que el submuestreo perdió.

La red recibe un canal por vóxel y en HyperNCD se le retira la capa final de clasificación, por lo que entrega 96 valores por vóxel.

Produce el descriptor semántico f_{sem} (Sección 2.2.1.1). Se entrena con las clases conocidas.

2.2.9 Muestreo por Rejilla e Índice Inverso

El muestreo por rejilla es la discretización que la rutina de preparación de Sonata aplica a la nube antes del codificador, y el índice inverso, la tabla que permite deshacerla. La nube se desplaza para centrarla en planta y apoyarla en $z = 0$, se divide en celdas de 2 cm por defecto y de cada celda ocupada se conserva un punto. El color se lleva al rango de 0 a 1. Los nueve valores de cada punto conservado, posición, color y normal, forman la entrada del codificador. El índice inverso anota, para cada punto de la entrada, la celda en la que cayó.

Con el índice inverso, el descriptor de cada celda se devuelve a todos los puntos de S que cayeron en ella, de modo que f_{ssl} queda definido sobre los mismos puntos que entregó el Módulo 1. Como la celda de 2 cm es menor que el vóxel de 5 cm del preprocesamiento, casi todos los puntos quedan solos en su celda.

2.2.10 Propagación Inversa del Submuestreo

También llamada *up-cast*, devuelve las características del codificador a la resolución de entrada reutilizando los índices de agrupamiento que el propio submuestreo registró. Es necesaria porque el codificador entrega su salida submuestreada varias veces. En los dos primeros pasos, desde la etapa más gruesa, las características se concatenan con las de la etapa anterior, la 5 con la 4 y el resultado con la 3, y en los restantes solo se copian, lo que da las 1088 dimensiones de f_{ssl}.

2.2.11 Caja de Sección y Corte

La caja de sección (*section box*) es un paralelepípedo que se ajusta sobre la nube para mostrar solo los puntos que quedan dentro; el corte es un plano que oculta todo lo que queda a un lado. Ambos permiten inspeccionar el interior de un edificio sin que el resto lo tape.

2.2.12 Reconstrucción de Superficie de Poisson

Método que convierte una nube de puntos con normales en una malla cerrada: busca la superficie cuya orientación mejor coincide con las normales de los puntos. La malla resultante puede exportarse como sólido en formatos de modelado 3D como OBJ, PLY o STL. Se usa para exportar un elemento ya segmentado.

2.3 Algoritmos

Esta sección reúne los procedimientos con que se descubren las clases novedosas y las arquitecturas de los dos métodos que el trabajo combina.

2.3.1 Descubrimiento de Clases Novedosas (NCD)

El NCD consiste en producir una asignación $\hat{y} : P \to \mathcal{C}$ correcta sobre $\mathcal{C}_k$ y consistente sobre $\mathcal{C}_n$, entendiendo por consistente que los puntos de una misma clase novedosa reciban la misma etiqueta, aunque esa etiqueta no tenga nombre. La etiqueta que recibe el punto i se denota $\hat{y}_i$.

2.3.2 Región o Superpunto

Una región es una agrupación de puntos vecinos que el método trata como una sola unidad. La implementación de HyperNCD forma las regiones con DBSCAN, un algoritmo de agrupamiento por densidad. Dos puntos pertenecen al mismo grupo si se puede ir de uno a otro pasando por puntos separados por menos de un radio dado, que es 0,5 m en la configuración de SemanticPOSS; los puntos sin vecinos dentro de ese radio quedan fuera de toda región. Se aplica sobre las coordenadas de los puntos de las clases novedosas. Los descriptores F_i de los puntos de una región se promedian en uno solo. Razonar sobre regiones y no sobre puntos reduce el número de nodos del hipergrafo en varios órdenes de magnitud.

2.3.3 Prototipo

Vector P_k que representa a un grupo de puntos o regiones semejantes. Hay K prototipos, tantos como clases. Una cabeza convolucional asigna a cada punto una puntuación $z_{i,k}$ por prototipo y una función *softmax* la convierte en pesos de pertenencia:

$$s_{i,k} = \frac{\exp(z_{i,k})}{\sum_{j=1}^{K} \exp(z_{i,j})}, \qquad \sum_{k=1}^{K} s_{i,k} = 1. \tag{II.11}$$

Convierte las puntuaciones de un punto en pesos de pertenencia. Los pesos de un punto suman 1 y se leen como un reparto. La matriz S_p de todos los $s_{i,k}$ es la *asignación suave*. Un punto puede pertenecer en parte a varios prototipos, lo que ocurre en las transiciones entre elementos arquitectónicos. Cada prototipo es el promedio de los descriptores, ponderado por esa pertenencia:

$$P_k = \frac{\sum_i s_{i,k} F_i}{\sum_i s_{i,k}}. \tag{II.12}$$

Así cada prototipo resume a los puntos que más le pertenecen.

2.3.4 Hipergrafo

Estructura $\mathcal{H} = (\mathcal{V}, \mathcal{E})$ en la que una arista puede conectar más de dos nodos a la vez. Frente a un grafo ordinario, restringido a relaciones binarias, permite expresar que varias clases conocidas concurren simultáneamente en la explicación de una novedosa.

2.3.4.1. Hiperarista

Elemento $e_i \in \mathcal{E}$ formado por una región r_i (Sección 2.3.2) y sus M regiones más afines según la afinidad S_b. Los nodos del hipergrafo son por tanto regiones y no prototipos. Como una sección contiene muchas más regiones que clases, cada hiperarista conecta un subconjunto selectivo de nodos, lo que no ocurriría con solo K prototipos. Los autores de HyperNCD reportan $M = 8$ como valor óptimo. Cada región recibe después el promedio de los descriptores de su hiperarista, ponderado por la afinidad, y es ese descriptor enriquecido el que se compara con los prototipos.

2.3.4.2. Afinidad entre Prototipos

Medida de semejanza entre dos regiones que combina un criterio semántico y uno geométrico. El semántico es la similitud por coseno entre sus descriptores,

$$S_s^{(ij)} = \frac{\langle F^{(i)},\, F^{(j)} \rangle}{\|F^{(i)}\|\, \|F^{(j)}\|}, \tag{II.13}$$

que vale 1 cuando ambos vectores apuntan en la misma dirección. El geométrico es un núcleo gaussiano,

$$S_g^{(ij)} = \exp\left(-\frac{\|g^{(i)} - g^{(j)}\|^2}{\tau} \right), \tag{II.14}$$

que vale 1 cuando las regiones tienen la misma forma y posición y cae hacia 0 al alejarse. El descriptor $F^{(i)}$ es por donde entra f_{ssl}. El rasgo $g^{(i)}$ es el f_{geo} de la región (Sección 2.2.1.2): aunque sale del descriptor F_i en el modelo propuesto, se sigue calculando para la afinidad y es igual en la línea base y en el modelo propuesto, con lo que el razonamiento geométrico del hipergrafo se conserva íntegro. Ambos se combinan en

$$S_b = \alpha\, S_g + (1 - \alpha)\, S_s, \qquad \alpha \in [0, 1], \tag{II.15}$$

donde α reparte el peso. Con $\alpha = 1$ la afinidad depende solo de la forma y con $\alpha = 0$, solo del significado. En el modelo propuesto el significado ya incluye el conocimiento auto-supervisado de f_{ssl}, mientras que la forma sigue medida por g. Así f_{ssl} entra una sola vez en el sistema y α conserva su interpretación.

2.3.5 Pseudo-etiqueta

Etiqueta que el propio modelo asigna a un punto sin supervisión humana y que después usa como si fuera verdadera para continuar entrenando. Se suaviza con $\epsilon = 0{,}15$: en lugar de certeza total, la clase ganadora recibe $1 - \epsilon$ y ϵ se reparte entre las demás, lo que reduce el sobreajuste a los propios errores. Ese valor es el que publica el artículo; la implementación ejecutada no lo expone como parámetro.

2.3.6 Normalización de Sinkhorn-Knopp

Procedimiento de normalización iterativa de una matriz que fuerza a que sus filas y columnas sumen valores prefijados. Aplicado a la asignación de pseudo-etiquetas, las reparte de forma equilibrada entre clases y evita el colapso, es decir, que casi todos los puntos caigan en la clase más frecuente.

2.3.7 Arquitectura de HyperNCD

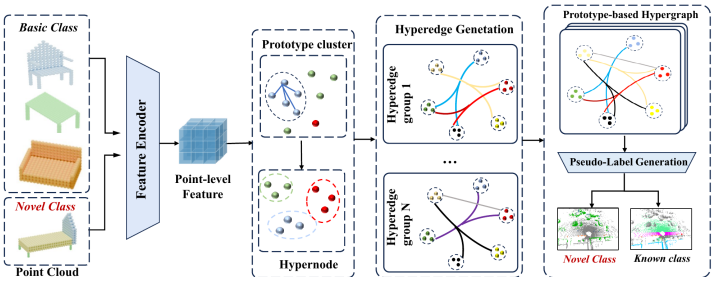


FIGURA 2.1: Arquitectura de HyperNCD. Fuente: Zhang et al. [13], Fig. 2.

HyperNCD [13] resuelve el NCD en cuatro etapas, que la Figura 2.1 presenta de izquierda a derecha:

1. **Codificador de características** (*Feature Encoder*). La nube, con puntos de clases conocidas y novedosas mezclados, entra a una red troncal que asigna a cada punto un vector de características (*point-level feature*). En la implementación es la MinkowskiUNet-34C que produce f_{sem}.
2. **Agrupamiento en prototipos** (*Prototype cluster*). Una convolución unidimensional seguida de *softmax* reparte los puntos entre K prototipos (Sección 2.3.3); cada prototipo pasa a ser un nodo del hipergrafo (*hypernode*).
3. **Generación de hiperaristas** (*Hyperedge generation*). Cada prototipo se une a sus M más afines según la afinidad S_b de la Ec. (II.15), que combina parecido geométrico y semántico. En cada lote se forma un grupo de hiperaristas.
4. **Hipergrafo de prototipos y pseudo-etiquetas** (*Prototype-based hypergraph*). Las hiperaristas se reajustan a medida que llegan lotes nuevos. Un prototipo cuya afinidad con uno conocido supera un umbral recibe la etiqueta de esa clase; si no, se considera novedoso y recibe una etiqueta nueva. Esas pseudo-etiquetas vuelven al entrenamiento.

La figura corresponde a la formulación del artículo. La implementación publicada que este trabajo ejecuta realiza las tres primeras etapas, pero no el reajuste dinámico de la cuarta, y asigna las pseudo-etiquetas con Sinkhorn-Knopp en lugar del umbral (Sección 1.5). Además, en la implementación los nodos del hipergrafo son regiones y no prototipos (Sección 2.3.4.1); este trabajo adopta esa formulación.

2.3.8 Arquitectura de Sonata

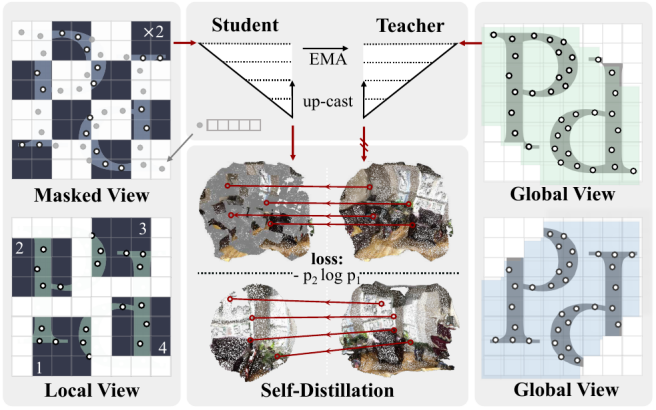


FIGURA 2.2: Marco de auto-destilación de Sonata. Fuente: Wu et al. [7], Fig. 5.

Sonata [7] no es un clasificador, sino un método para preentrenar un codificador sin etiquetas. La Figura 2.2 muestra cómo se entrenó. Este trabajo no repite ese entrenamiento; usa únicamente el codificador resultante, congelado. Del procedimiento interesan dos propiedades, porque explican qué contiene f_{ssl}:

- **Auto-destilación.** Dos copias de la misma red, estudiante y maestro, reciben vistas distintas de una escena, y el estudiante aprende a reproducir lo que el maestro dice de cada punto. Como el estudiante ve menos contexto que el maestro, para acertar tiene que aprender qué es el punto y no solo su geometría local.
- **Codificador sin decodificador y *up-cast*.** La red es un PTv3 al que se le quitó el decodificador, porque este favorecía el atajo geométrico (Sección 2.2.4); para conservar información de varias escalas, las características se devuelven a la resolución de las etapas anteriores mediante el *up-cast* (Sección 2.2.10).

2.4 Métricas de Evaluación de Modelos

Para evaluar la efectividad del modelo se utilizan las métricas siguientes.

2.4.1 Intersection over Union (IoU)

Para una clase c, la intersección sobre unión se define como:

$$\mathrm{IoU}_c = \frac{TP_c}{TP_c + FP_c + FN_c}, \tag{II.16}$$

Mide cuánto coinciden los puntos predichos de una clase con los reales: vale 1 si coinciden todos y 0 si no coincide ninguno.

2.4.2 IoU Media (mIoU)

La métrica agregada es el promedio de la anterior sobre un conjunto de clases:

$$\mathrm{mIoU}(\mathcal{S}) = \frac{1}{|\mathcal{S}|}\sum_{c\in\mathcal{S}} \mathrm{IoU}_c, \qquad \mathcal{S} \in \{\mathcal{C}_k,\, \mathcal{C}_n,\, \mathcal{C}\}. \tag{II.17}$$

Resume en un solo número el desempeño sobre todas las clases del conjunto. Se reporta por separado como mIoU *known*, mIoU *novel* y mIoU *all*.

2.4.3 Análisis de Dominancia

La Ec. (II.17) es una media aritmética no ponderada, por lo que

$$\frac{\partial\, \mathrm{mIoU}}{\partial\, \mathrm{IoU}_c} = \frac{1}{|\mathcal{S}|} \qquad \forall\, c \in \mathcal{S}. \tag{II.18}$$

Cada clase pesa igual con independencia de su número de puntos. Por ello el valor agregado se acompaña siempre de la clase dominante:

$$c^* = \arg\operatorname*{máx}_{c\in\mathcal{S}} \big|\mathrm{IoU}_c - \mathrm{mIoU}(\mathcal{S})\big|. \tag{II.19}$$

Identifica la clase que más se aparta del promedio y que, por tanto, más explica su valor.

2.4.4 Emparejamiento Húngaro

Como las clases de $\mathcal{C}_n$ se descubren sin etiqueta, la correspondencia entre agrupaciones predichas y clases reales se resuelve por asignación óptima:

$$\sigma^* = \operatorname*{arg\,máx}_{\sigma \in \mathfrak{S}_{|\mathcal{C}_n|}} \sum_{c \in \mathcal{C}_n} \mathrm{IoU}\big(\hat{y} = \sigma(c),\ y = c\big). \tag{II.20}$$

Entre todas las correspondencias posibles elige la de mayor IoU total, y así cada grupo descubierto recibe el nombre de una clase real.

2.4.5 Índice de Rand Ajustado (ARI)

Mide la coincidencia entre la agrupación descubierta y la real con independencia del emparejamiento:

$$\mathrm{ARI} = \frac{\sum_{ij} \binom{n_{ij}}{2} - \Big[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}\Big] / \binom{n}{2}}{\frac{1}{2}\Big[\sum_i \binom{a_i}{2} + \sum_j \binom{b_j}{2}\Big] - \Big[\sum_i \binom{a_i}{2} \sum_j \binom{b_j}{2}\Big] / \binom{n}{2}}, \tag{II.21}$$

Vale 1 cuando las dos agrupaciones coinciden y cerca de 0 cuando coinciden solo por azar.

2.4.6 Información Mutua Normalizada (NMI)

Complementa al ARI midiendo la información que una agrupación aporta sobre la otra:

$$\mathrm{NMI}(U, V) = \frac{2\, I(U; V)}{H(U) + H(V)}, \tag{II.22}$$

Vale 1 cuando conocer una agrupación determina la otra y 0 cuando son independientes.

2.4.7 Linear Probing

Protocolo de evaluación de un descriptor congelado, en el que se entrena únicamente un clasificador lineal W sobre él,

$$\hat{y}_i = \arg\max \left(W^\top f_{\text{ssl}}(x^i) + b \right), \tag{II.23}$$

que asigna a cada punto la clase con mayor puntuación. Como el descriptor está congelado, el desempeño resultante queda atribuido a la representación y no al clasificador.

2.4.8 Ganancia Absoluta

Diferencia de mIoU sobre las clases novedosas entre el modelo híbrido y la línea base:

$$\Delta\text{mIoU}^{\text{novel}} = \text{mIoU}^{\text{novel}}_{\text{hib}} - \text{mIoU}^{\text{novel}}_{\text{base}}. \tag{II.24}$$

Indica cuántos puntos de mIoU gana, o pierde, el modelo híbrido en las clases novedosas.

2.4.9 Brecha Cerrada

La ganancia absoluta no es interpretable por sí sola, porque el techo alcanzable depende de la sección. Esta métrica, propuesta en el presente trabajo, la normaliza por la brecha que separa a la línea base de su propio desempeño sobre las clases que sí vio etiquetadas:

$$G = \frac{\text{mIoU}^{\text{novel}}_{\text{hib}} - \text{mIoU}^{\text{novel}}_{\text{base}}}{\text{mIoU}^{\text{known}}_{\text{base}} - \text{mIoU}^{\text{novel}}_{\text{base}}}. \tag{II.25}$$

G es adimensional. $G = 0$ indica que la sustitución no aporta, $G = 1$ que las clases novedosas alcanzan la calidad con que el modelo segmenta las conocidas y $G < 0$ degradación.

2.4.10 Atribución por Etapas

Como el proceso es separable en dos etapas, la métrica se reporta también en el punto intermedio y no solo al final, con *linear probing* sobre el descriptor (Ec. (II.23)) antes de la formación de prototipos y mIoU (Ec. (II.17)) después. Esa separación permite atribuir una mejora a la representación o al razonamiento, y no al sistema tratado como caja negra.

2.5 Diseño de la Tesis

La Figura 2.3 organiza el sistema en cinco módulos; sobre cada flecha se indica lo que pasa de un módulo al siguiente. El diagrama muestra solo imágenes y fórmulas. Cada fórmula enlaza con su ecuación en el marco teórico, donde se explica, y allí cada símbolo enlaza con su fila en la nomenclatura del Anexo B. El diagrama representa el diseño propuesto. Las flechas continuas son conexiones que el código ya realiza y las discontinuas, conexiones diseñadas y todavía no implementadas (Sección 2.5.6).

2.5.1 Módulo 1: Preprocesamiento

Recibe los archivos LAS de una sección y entrega la matriz S de nueve columnas (Ec. (II.8)). No usa ningún modelo y se ejecuta una vez, en CPU.

1. Lectura por bloques del archivo LAS (Sección 2.1.3).
2. Extracción de las coordenadas (x, y, z) y del color (R, G, B), con el color llevado al rango 0 a 255 (Sección 2.1.3).
3. Centrado de las coordenadas (Ec. (II.2)).
4. Filtrado de puntos aislados por SOR, con $q = 20$ y $\lambda = 2$ (Ec. (II.4)).
5. Voxelización a 5 cm, con un representante por vóxel (Ec. (II.5)).
6. Estimación de normales por vecindad, con $q = 20$ (Ec. (II.7)).
7. Escritura de S y de sus metadatos (Sección 2.1.9).

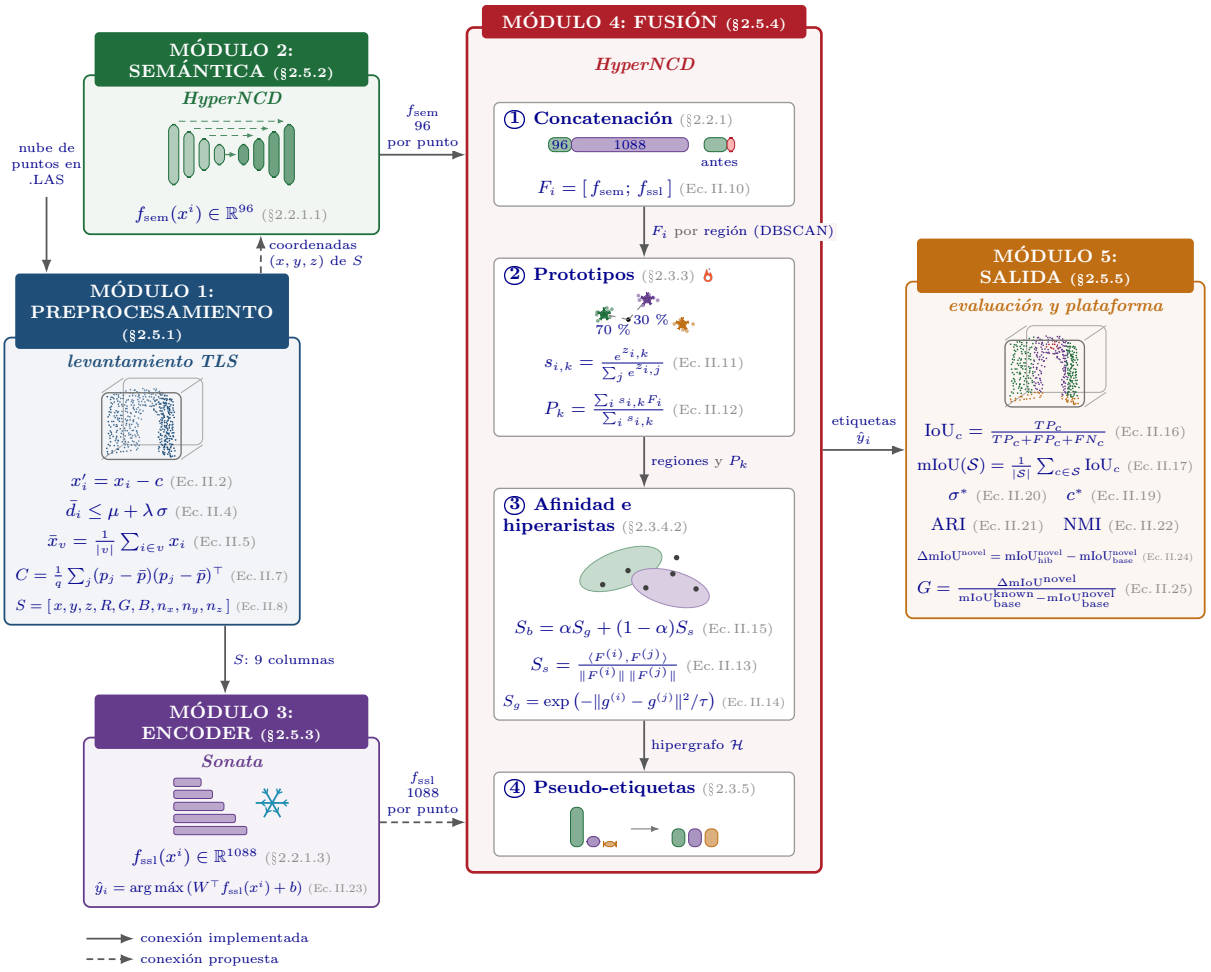


FIGURA 2.3: Diseño del modelo híbrido propuesto. Flecha continua: conexión implementada; discontinua: propuesta. Elaboración propia.

El SOR precede a la voxelización para que los puntos espurios no se promedien con los válidos, y las normales se estiman al final, sobre los representantes. Los metadatos registran los parámetros usados, el centro restado y el número de puntos tras cada paso.

La Tabla 2.1 separa los archivos que forman parte del sistema de los que se generaron para examinar y documentar el dato. Estos últimos no son etapas de la arquitectura y por eso no figuran en el diagrama.

Tabla 2.1: Archivos del sistema y archivos de inspección, para los escaneos de prueba. Elaboración propia.

Archivo	Función
Del sistema	
`PL7_SCN0001.las`	Entrada del Módulo 1: escaneo en bruto
`PL7_SCN0001_preprocesado.npy`	Salida del Módulo 1: matriz S, entrada del Módulo 3
`PL7_SCN0001_preprocesado.json`	Metadatos del Módulo 1: parámetros, centro restado y conteos
`PL13_feats.npz`	Salida del Módulo 3 para PL13: f_{ssl}, coordenadas e índice inverso
De inspección y documentación	
`PL7_muestra_100.csv`	Cien puntos del archivo LAS en texto, para examinar sus campos
`PL7_preprocesado_muestra_100.csv`	Cien filas de S en texto, para examinar sus columnas
`PL7_SCN0001_preprocesado.ply`	Copia de S para verla en un visor 3D
`inspeccionar_data.py`	Guion que comprueba la coherencia entre los archivos anteriores, sin ejecutar ningún modelo
`inspeccion_DATA.json`	Reporte que produce ese guion

2.5.2 Módulo 2: Semántica

Recibe las coordenadas de S y entrega $f_{\text{sem}}(x^i)$, 96 números por punto (Sección 2.2.1.1). El color y la normal de S no entran en este módulo.

1. Cuantiza las coordenadas en una rejilla dispersa de 5 cm y asigna a cada vóxel ocupado la característica constante 1 (Sección 2.2.6).
2. Aplica la red MinkowskiUNet-34C, de convoluciones dispersas, con codificador, decodificador y conexiones de salto (Secciones 2.2.7 y 2.2.8).
3. Entrena con las clases conocidas (Sección 2.1.11).
4. Se reentrena en cada época (Sección 2.2.1.1).

2.5.3 Módulo 3: *Encoder*

Recibe S con sus nueve columnas y entrega $f_{\text{ssl}}(x^i)$, 1088 números por punto (Sección 2.2.1.3).

1. Muestrea la sección en una rejilla propia de 2 cm y forma la entrada de nueve valores por punto (Sección 2.2.9).

2. Aplica el PTv3 de Sonata, preentrenado de forma auto-supervisada (Secciones 2.2.2 y 2.3.8). El PTv3 ordena los puntos por serialización (Sección 2.2.5) y no usa decodificador, para evitar el atajo geométrico (Sección 2.2.4).
3. Devuelve la salida a la resolución de entrada mediante *up-cast* (Sección 2.2.10).
4. Se usa congelado, una vez por sección, y guarda en disco los descriptores, las coordenadas y el índice inverso (Secciones 2.2.3 y 2.2.9).
5. Su calidad se mide con *linear probing* (Sección 2.4.7).

2.5.4 Módulo 4: Fusión

Recibe f_{sem} y f_{ssl} y entrega la etiqueta $\hat{y}_i$ de cada punto (Sección 2.3.1). Es el único módulo que este trabajo modifica. Realiza cinco operaciones. La figura las muestra en cuatro recuadros y coloca el agrupamiento en regiones sobre la flecha que une el primero con el segundo.

1. Concatenación de los descriptores en F_i (Ec. (II.10)).
2. Agrupamiento en regiones mediante DBSCAN (Sección 2.3.2).
3. Formación de prototipos (Ecs. (II.11) y (II.12)).
4. Construcción del hipergrafo por afinidad (Ec. (II.15) y Sección 2.3.4.1).
5. Asignación de pseudo-etiquetas equilibradas (Secciones 2.3.5 y 2.3.6).

2.5.5 Módulo 5: Salida

Recibe $\hat{y}_i$, mide su calidad y la pone a disposición del ingeniero en una plataforma.

Evaluación

1. Segmentación semántica por punto (Sección 2.1.10).
2. Emparejamiento húngaro de los grupos nuevos (Sección 2.4.4).
3. IoU y mIoU sobre clases conocidas, novedosas y total (Secciones 2.4.1 y 2.4.2).
4. Clase dominante del mIoU (Sección 2.4.3).
5. ARI y NMI (Secciones 2.4.5 y 2.4.6).
6. ΔmIoU frente a la línea base (Sección 2.4.8).
7. Brecha cerrada G (Sección 2.4.9).
8. Atribución por etapas (Sección 2.4.10).

Plataforma

9. Caja de sección y cortes (Sección 2.2.11).
10. Lista desplegable de elementos: columnas, vigas, techos y las clases novedosas.
11. Al hacer clic en un elemento, sus puntos se resaltan en la vista.
12. Exportación del elemento como sólido 3D (Sección 2.2.12).

2.5.6 Diseño Propuesto e Implementación Actual

La Tabla 2.2 indica, por módulo, qué está implementado y qué parte es estado del arte y qué parte es aporte propio. Cada módulo vale 100 %; el aporte propio previsto suma 270 de 500, el 54 %. El escaneo de prueba no trae etiquetas y su anotación está pendiente.

Tabla 2.2: Implementación actual y aporte previsto por módulo. Elaboración propia.

Módulo	**Implementación actual**	**Estado del arte**	**Aporte propio**
1. Preprocesamiento	Implementado; probado en PL7 y ejecutado sobre los 23 archivos en Khipu	20 %	80 %
2. Semántica	Código de HyperNCD; falta el lector de S	90 %	10 %
3. *Encoder*	Extracción implementada y probada en PL13; falta medir su calidad	60 %	40 %
4. Fusión	Por implementar	30 %	70 %
5. Salida	mIoU disponible; el resto, por implementar	30 %	70 %
Total		230	270

CAPÍTULO III

MARCO METODOLÓGICO

Este capítulo explica el procedimiento del proyecto con el detalle necesario para repetirlo. Primero presenta el diagrama de flujo general, el hardware y el repositorio donde está el código. Después describe cada unidad de trabajo: qué se ejecuta, con qué entrada y qué salida produce. La Unidad 1 y la rama auto-supervisada de la Unidad 2 ya se ejecutaron sobre escaneos reales y se describen con sus resultados medidos, de modo que la metodología se corresponde con los resultados preliminares. El resto de la Unidad 2 y la Unidad 3 describen el procedimiento previsto.

3.1 Diagrama de Flujo General

La Figura 3.1 muestra los pasos del proyecto en el orden en que se ejecutan, de P0 a P5. Las secciones se procesan una a una, y en cada sección los pasos P2 y P3 se repiten tres veces (Sección 3.6.2).

El flujo muestra tres elementos que la arquitectura no representa. El primero es la frontera de disco. PTv3 requiere CUDA 12.4 y MinkowskiEngine CUDA 11.x (11.8 en el clúster), que no coexisten en un mismo proceso, por lo que f_{ssl} se escribe en un archivo `.npz` y se lee desde el otro entorno. El segundo es la asimetría de frecuencia, porque P2 se ejecuta una vez por sección y P3 en cada época, lo que justifica el almacenamiento intermedio. El tercero son los dos bucles, el de repeticiones y el de recorrido de secciones.

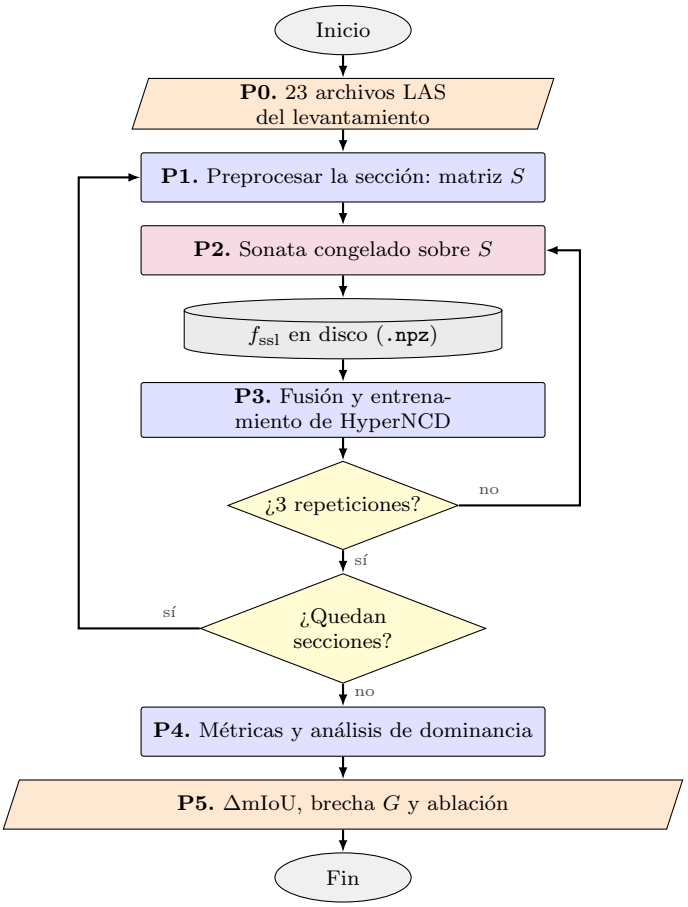


FIGURA 3.1: Flujo general del proyecto. Elaboración propia.

3.2 Hardware y Repositorio

Hardware El proyecto se ejecuta en el clúster Khipu de UTEC, cuyos recursos se detallan en la Tabla B. Cada paso pide al gestor de colas solo lo que necesita. El preprocesamiento usa 16 núcleos y 64 GB de memoria, sin GPU. La extracción de descriptores con Sonata usa una GPU RTX A6000, 8 núcleos y 64 GB. El entrenamiento de HyperNCD usa una GPU, 8 núcleos y 32 GB. La prueba preliminar de la Unidad 1 se ejecutó en una laptop con procesador AMD Ryzen 5 8645HS de 6 núcleos y 12 hilos, 16 GB de memoria y Windows 11 Home (versión 10.0.26200). Su GPU, una NVIDIA GeForce RTX 3050 de 6 GB, no interviene en el preprocesamiento. El programa corrió con Python 3.12.7, `laspy` 2.7.0, NumPy 2.3.5 y Open3D 0.20.0.

Repositorio Todo el código está en el repositorio del proyecto.[1] Contiene el programa de preprocesamiento (`preprocesar.py`), el programa que extrae y guarda los descriptores de Sonata (`extraer_features.py`), el código de HyperNCD y de Sonata en dos carpetas, la carpeta `experimentacion` con las herramientas para revisar y medir las salidas (el visor `visor_3d.py` y el programa `visualizar_features.py`), un guion de instalación que crea un entorno para cada método, porque sus dependencias son incompatibles, y un guion por paso para lanzarlo en el clúster. Los archivos LAS y las salidas no se guardan en el repositorio; se copian aparte.

3.3 Diseño General del Sistema

El diseño del sistema y el significado de cada módulo, de cada fórmula y de cada parámetro se describen en la Sección 2.5 y se resumen en la Figura 2.3. Esta sección no repite esa descripción. Fija las propiedades de ejecución que el diagrama no expresa, es decir, qué corre en paralelo, con qué frecuencia y con qué formato de intercambio.

Las dos ramas están en paralelo porque ninguna consume la salida de la otra: parten de la misma nube preparada y solo se encuentran en el Módulo 4. Esa independencia permite ejecutar el Módulo 3 una sola vez, guardar su salida y reutilizarla en todas las épocas del Módulo 2.

3.3.1 Estructura y Contrato de los Módulos

Cada conexión entre módulos transporta un objeto con una forma determinada. La Tabla 3.1 declara ese contrato para que cualquier incompatibilidad de dimensión o de resolución aparezca antes de implementar.

[1] https://github.com/manyorycuevamendoza/PFC1-HyperNCD-Sonata

Tabla 3.1: Contrato de los módulos. Elaboración propia.

Módulo	Entrada	Salida	Régimen	Frecuencia
1	Archivos LAS de la sección, N puntos	Matriz S de 9 columnas: coord., color, normal	Sin modelo	Una vez por sección
2	Coordenadas de S, con característica constante 1	f_{sem}, 96 dim. por punto, rejilla dispersa de 5 cm	Entrenado	Cada época
3	S, 9 columnas	f_{ssl}, 1088 dim. por punto, rejilla propia de 2 cm, con índice inverso	Congelado	Una vez por sección
4	f_{sem} y f_{ssl} alineados	Prototipos, hipergrafo y pseudo-etiquetas	Entrenado	Cada época
5	Pseudo-etiquetas y verdad de terreno	Etiqueta por punto, mIoU por clase e interfaz de consulta	Evaluación y consulta	Por corrida

Las salidas de los Módulos 2 y 3 no viven en la misma rejilla. Se concilian con el índice inverso (Sección 2.2.9), que se almacena junto con las características para que el Módulo 4 reciba los dos descriptores alineados punto a punto.

3.3.2 Herramientas por Módulo

Las herramientas, el módulo en que interviene cada una y los entornos con sus versiones se recogen en la Tabla A.1 del Anexo A.

3.3.3 Visor 3D del Proyecto

Para revisar las nubes en cada etapa se construyó un visor propio, el programa `visor_3d.py` de la carpeta `experimentacion` del repositorio. Recibe un escaneo en sus distintas etapas y genera una página web que se abre en el navegador sin instalar nada. La página tiene una sola escena 3D y un botón por etapa. Las etapas son la entrada cruda, los puntos que descarta el filtrado, la matriz S de la Unidad 1 con su color y con sus normales, y los descriptores de Sonata vistos de tres formas. Todas las etapas comparten las coordenadas, la escala y la cámara. Al cambiar de botón

el ángulo no se mueve, así que las etapas se comparan desde el mismo punto de vista. La página permite girar, acercar y exportar la vista a PNG. El visor también puede recortar un cubo y dibujar todos sus puntos. Todas las figuras de nubes de este capítulo se obtuvieron así.

El visor también comprueba que los datos de cada etapa correspondan entre sí. Los descriptores de Sonata se asocian a los puntos de S por el orden de las filas, y el visor verifica que cada punto caiga en su celda antes de dibujar. Si no corresponden, se detiene con un aviso, como se explica en la Sección 3.4.3.

3.4 Unidad 1. Preprocesamiento

La primera unidad convierte un escaneo en bruto, en formato LAS, en la matriz S que entra a los modelos de la Unidad 2. Corresponde al Módulo 1, descrito en la Sección 2.5.1, y está en el programa `preprocesar.py` del repositorio. Se describe tal como se ejecutó en la prueba preliminar, sobre el escaneo `PL7_SCN0001.las` de la Catedral de Lima, en el equipo local de la Sección 3.2. Ese escaneo es una parte pequeña del levantamiento, que se describe en la Sección 3.4.1.

3.4.1 Datos de Entrada y Ejecución

El levantamiento. La Catedral de Lima se recibe como 23 archivos LAS, que suman 20,05 GB y 589 708 290 puntos. Cada archivo es un escaneo de terreno independiente y trae, por punto, coordenadas UTM, color de 16 bits, intensidad, número de retorno y clase, de los cuales solo se usan la posición y el color (Sección 2.1.3). Por término medio un archivo tiene unos 25,6 millones de puntos y ocupa unos 0,87 GB; PL7, el de la prueba, tiene 8 590 904 puntos, el 1,46 % del total. Los archivos no traen etiquetas.

Cómo entran los archivos. La unidad de trabajo es la sección (Sección 2.1.9). El programa `preprocesar.py` acepta uno o varios archivos LAS en `--input` y los une en una sola sección. En el levantamiento completo se usó un archivo por sección, de modo que hay 23 secciones y cada una produce su propia pareja de archivos, la matriz S en `.npy` y los metadatos en `.json`. Un archivo se lee por bloques de cinco millones de puntos, pero la sección completa se acumula en memoria antes del filtrado, por lo que debe caber en la RAM del nodo.

Paralelismo. Las secciones son independientes, ninguna usa datos de otra. El programa no reparte una sección entre procesos ni lee varios archivos a la vez. El guion de lanzamiento `preprocesar.sbatch` recorre los archivos de uno en uno dentro de un solo trabajo del clúster, con 16 núcleos, 64 GB de memoria, sin GPU y un límite de cuatro horas. El único paralelismo es el interno de las operaciones de Open3D sobre esos 16 núcleos. Si un archivo ya tiene su `.npy`, el guion lo salta, de modo que un trabajo interrumpido se retoma sin repetir lo hecho. Como las secciones son independientes, también podrían lanzarse como trabajos separados, uno por archivo. Eso no se ejecutó, porque los 23 archivos tardaron en total unos 18 minutos.

Comandos en el clúster. Los comandos para lanzar cada paso en Khipu, revisar los descriptores y generar el visor están en el Anexo A. Cada trabajo escribe su registro en `logs/`, con los conteos tras cada paso, que son los que se comparan al verificar una repetición (Sección 3.4.3).

3.4.2 Procedimiento

El programa recibe un archivo LAS y entrega la matriz S. Realiza seis pasos, en este orden. La Figura 3.2 los resume con los conteos de la prueba. Cada paso se describe a continuación con su resultado en la prueba. Los pasos cortos llevan

pseudocódigo y los largos, diagrama de flujo. Los parámetros son tres, el tamaño de vóxel $\ell = 5\,\text{cm}$, el número de vecinos $q = 20$ y el umbral del filtrado $\lambda = 2$. La prueba tardó 14,4 segundos en total.

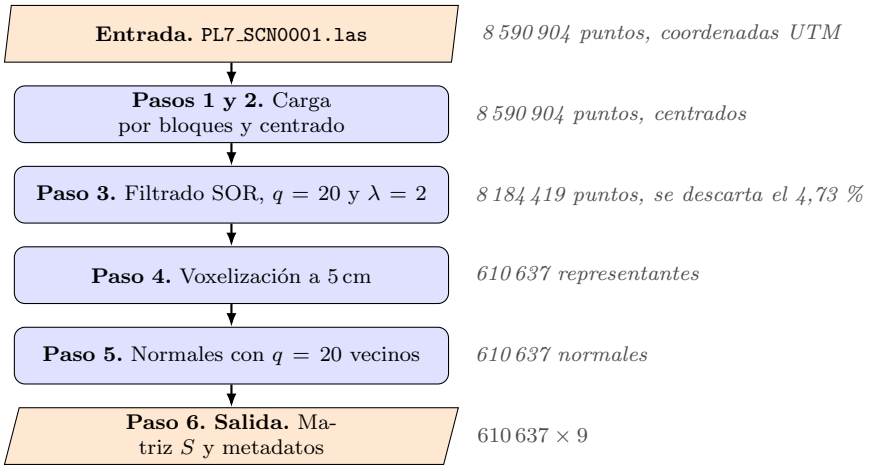


FIGURA 3.2: Flujo del preprocesamiento en PL7. Elaboración propia.

Paso 1. Carga

El archivo se abre con `laspy` y se lee en bloques de cinco millones de puntos, para no pedir toda la memoria de una vez, como se explica en la Sección 2.1.3. De cada punto se conservan solo la posición (x, y, z) y el color (R, G, B), que son los datos que usan los modelos. La intensidad, el número de retorno y la clase se descartan. El color viene en 16 bits, con un máximo de 65 535, y se divide entre 256 para llevarlo al rango de 0 a 255. En la prueba se leen 8 590 904 puntos en dos bloques, en cerca de un segundo. Las figuras de PL7 dibujan 200 000 puntos tomados al azar. La Figura 3.3 muestra el escaneo cargado.

Algoritmo 1: Paso 1 del Módulo 1, carga.

Entrada: archivo LAS de N puntos
Salida: posiciones $P \in \mathbb{R}^{N\times 3}$ y colores $C \in [0, 255]^{N\times 3}$
para cada *bloque de* 5×10^6 *puntos del archivo* **hacer**
    añadir a P las columnas x, y y z del bloque
    añadir a C las columnas R, G y B del bloque
si *el color usa 16 bits* **entonces** $C \leftarrow C/256$

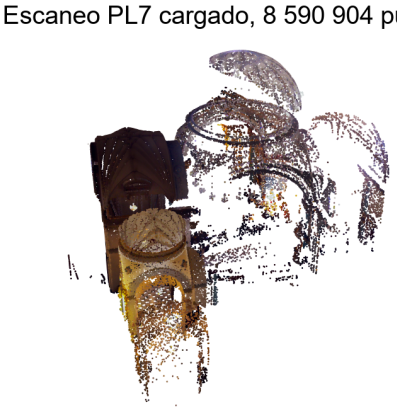


FIGURA 3.3: PL7 tras la carga. Elaboración propia.

Paso 2. Centrado

Las coordenadas vienen en metros sobre el mapa, en el sistema UTM, con valores de cientos de miles y de millones. Con números tan grandes se pierde precisión al guardarlos en precisión simple, así que a todos los puntos se les resta el centro de la caja que los contiene, según la Ecuación II.2. La forma de la nube no cambia, solo se traslada al origen. El centro restado se guarda en los metadatos para poder devolver cualquier resultado a su posición real. La Tabla 3.2 muestra el efecto en la prueba. Este paso no lleva imagen porque la nube se ve igual antes y después, solo cambian los números de los ejes.

Algoritmo 2: Paso 2 del Módulo 1, centrado.

Entrada: posiciones P en coordenadas UTM
Salida: posiciones centradas y centro c
$c \leftarrow (\min P + \max P)/2$, calculado por eje
$P \leftarrow P - c$
guardar c para los metadatos

Tabla 3.2: Coordenadas de PL7 antes y después del centrado, en metros. Elaboración propia.

Eje	Antes, en UTM	Centro	Después
x	278 775,30 a 278 826,64	278 800,97	−25,67 a 25,67
y	8 667 724,83 a 8 667 780,02	8 667 752,42	−27,60 a 27,60
z	159,36 a 188,83	174,10	−14,73 a 14,73

Paso 3. Filtrado SOR

El escaneo trae puntos sueltos que no pertenecen a ninguna superficie. Para cada punto se mide la distancia media a sus 20 vecinos más cercanos. Si esa distancia supera la media de toda la nube en más de dos desviaciones estándar, el punto se descarta, según la Ecuación II.4. El filtrado se hace antes de voxelizar para que esos puntos no se promedien con los válidos. En la prueba quedan 8 184 419 puntos y se descartan 406 485, el 4,73 %. El filtrado tardó cerca de doce segundos, la mayor parte de la prueba.

La Figura 3.5 marca en rojo los puntos descartados, que se concentran en las zonas ralas, lejos del escáner.

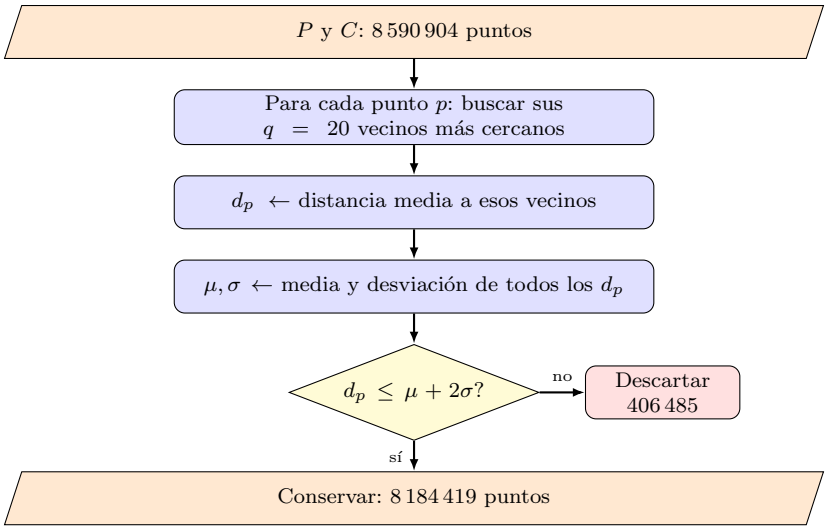


FIGURA 3.4: Filtrado SOR. Elaboración propia.

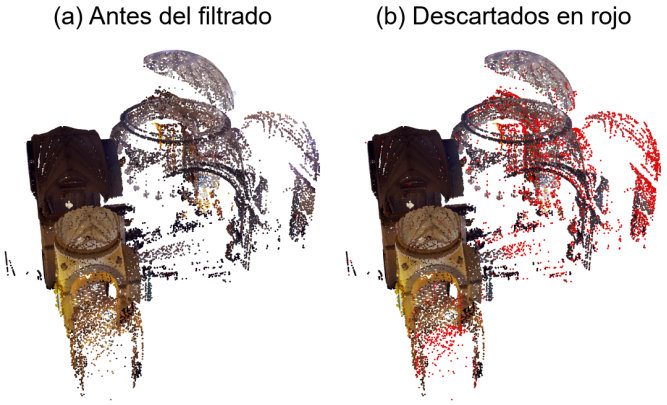


FIGURA 3.5: PL7 con los puntos descartados por el SOR en rojo. Elaboración propia.

Paso 4. Voxelización

El escáner mide muchos más puntos cerca de él que lejos. Para igualar la densidad, el espacio se divide en cubos de 5 cm de lado y los puntos de cada cubo se sustituyen por uno solo, con su posición y su color promedio, según la Ecuación II.5. No se pierde ninguna superficie, cambia cuántos puntos la describen. En la prueba quedan 610 637 representantes, el 7,46 % de los puntos que salieron del filtrado. La Figura 3.7 muestra el escaneo completo antes y después, y la Figura 3.8 muestra de

cerca un cubo de 1 m de lado. En ese cubo los 48 098 puntos de la entrada quedan en 921 representantes. La entrada contiene varias superficies paralelas separadas por pocos centímetros, que en la salida ya no se distinguen. Es el límite del vóxel de 5 cm, que no separa superficies más próximas que su lado. En el cubo se dibujan todos los puntos, con el color aclarado por un factor de 2,2.

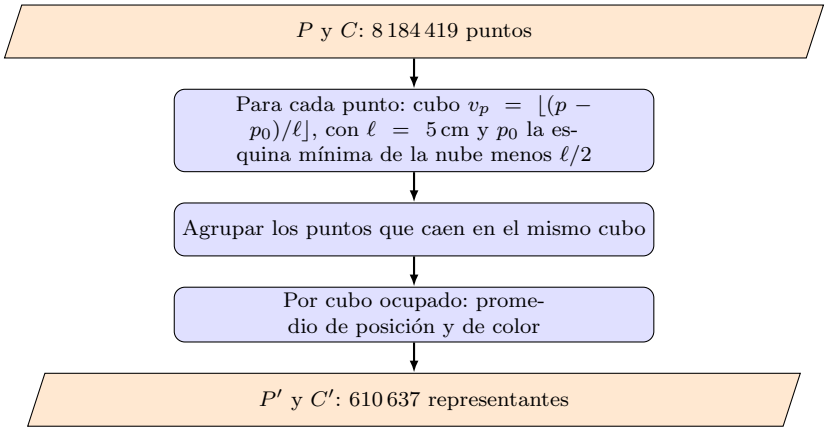


FIGURA 3.6: Voxelización. Elaboración propia.

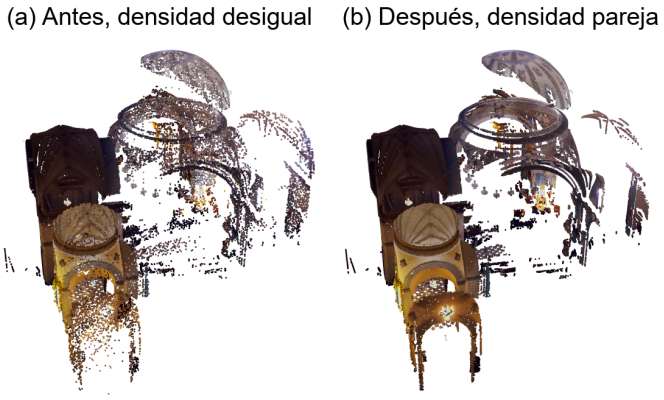


FIGURA 3.7: PL7 antes y después de la voxelización. Elaboración propia.

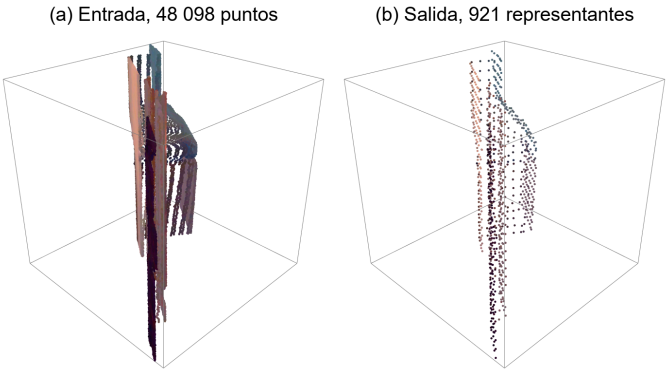


FIGURA 3.8: Cubo de 1 m de PL7 antes y después de la voxelización. Elaboración propia.

Paso 5. Normales

La normal indica hacia dónde mira la superficie en cada punto, y el escáner no la mide. Se estima con los 20 vecinos de cada representante, según la Ecuación II.7. La dirección en la que esos vecinos menos se dispersan es la perpendicular a la superficie. Todas las normales se orientan con $n_z \geq 0$ para que sean consistentes entre sí. En la Figura 3.10 cada normal se usa como color, de modo que las superficies con la misma orientación toman el mismo tono. En las bóvedas el color cambia de forma gradual, siguiendo su curvatura. En las zonas ralas, lejos del escáner, aparecen manchas de colores mezclados, porque los 20 vecinos de un punto aislado no describen bien la superficie.

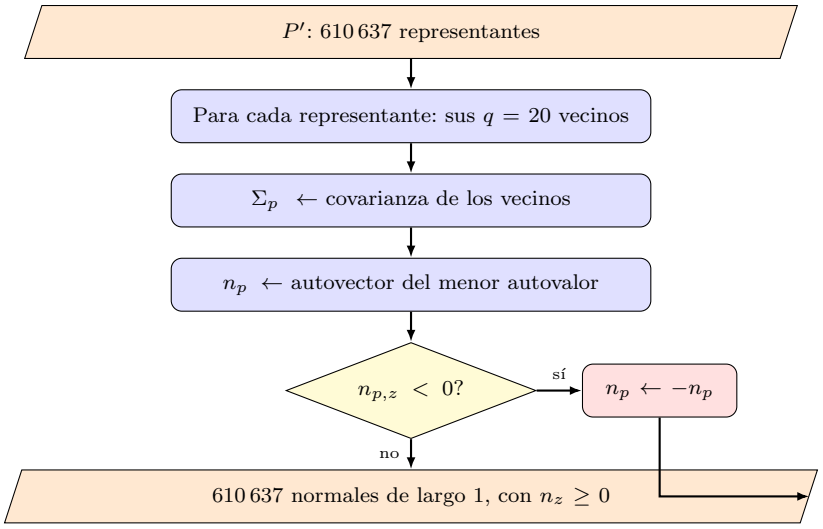


FIGURA 3.9: Estimación de normales. Elaboración propia.

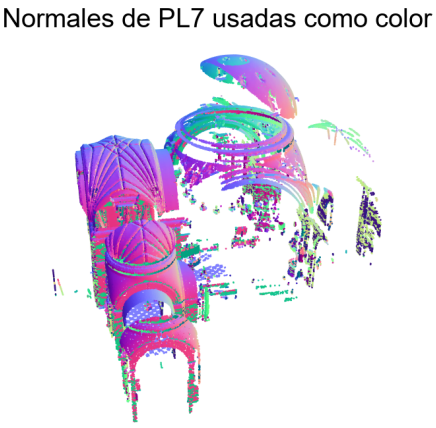


FIGURA 3.10: Normales de PL7 como color. Elaboración propia.

Paso 6. Escritura

Se guarda la matriz S, descrita en la Ecuación II.8, con una fila por representante y sus nueve valores, la posición, el color y la normal. Se guarda en precisión simple, y en la prueba ocupa 22 MB con 610 637 filas. Junto a ella se escribe un archivo de metadatos con los parámetros usados, el centro restado y el número de puntos tras cada paso. La Tabla 3.3 muestra su contenido en la prueba.

Algoritmo 3: Paso 6 del Módulo 1, escritura.

Entrada: P', C', n, centro c y conteos de cada paso
Salida: archivo de la matriz S y archivo de metadatos
$S \leftarrow [\, P' \mid C' \mid n \,]$ en precisión simple
guardar S
guardar ℓ, q, λ, c y los conteos en el archivo de metadatos

Tabla 3.3: Metadatos de PL7. Elaboración propia.

Campo	Valor	Significado
`voxel`	0,05	Lado del vóxel, en metros
`q`	20	Vecinos del filtrado y de las normales
`lambda`	2	Umbral del filtrado
`offset_utm`	278 800,97, 8 667 752,42 y 174,10	Centro restado en el Paso 2
`n_crudo`	8 590 904	Puntos tras la carga
`n_sor`	8 184 419	Puntos tras el filtrado
`n_voxel`	610 637	Representantes tras la voxelización
`segundos`	14,4	Tiempo total

3.4.3 Verificación y Reproducibilidad

El programa no tiene pasos aleatorios, de modo que la misma entrada produce la misma salida. La prueba se ejecutó con la orden siguiente, con los valores por defecto del programa.

```
python preprocesar.py --input PL7_SCN0001.las \
       --output PL7_SCN0001_preprocesado \
       --voxel 0.05 --q 20 --lam 2
```

Tardó 14,4 segundos en el equipo local de la Sección 3.2. Una repetición se comprueba comparando los conteos del archivo de metadatos, `n_crudo`, `n_sor` y `n_voxel`, con los de la Tabla 3.3. Deben coincidir exactamente.

Lo que se repite son los puntos, no su orden de filas. Como los descriptores de Sonata se asocian a los puntos por ese orden, cada archivo de descriptores debe usarse con la misma matriz S que entró a Sonata, y no con otra repetición del preprocesamiento.

El mismo programa preprocesó después el levantamiento completo en el clúster Khipu, con los mismos parámetros. Los 23 archivos LAS pasaron de 589 708 290 puntos a 17 807 859 representantes. La suma de los tiempos por archivo fue de 1 079 segundos, unos 18 minutos. Los conteos de PL7 en esa corrida coinciden con los de la prueba local.

3.5 Unidad 2. Extracción de Descriptores y Fusión

La segunda unidad abarca los Módulos 2, 3 y 4. Las dos ramas de extracción producen, sobre la misma sección preparada, los dos descriptores que el punto de fusión concatena.

3.5.1 Rama Semántica

La rama semántica (Módulo 2, Sección 2.5.2) es un módulo entrenado, por lo que se ejecuta en cada época. Aún no se ha ejecutado.

3.5.2 Rama Auto-supervisada Congelada

La rama auto-supervisada corresponde al Módulo 3 (Sección 2.5.3) y está en el programa `extraer_features.py` del repositorio. Se ejecuta una sola vez por sección y su salida se escribe en disco para reutilizarla en todas las épocas.

Esta rama ya se ejecutó en una prueba preliminar sobre el escaneo PL13, en el clúster Khipu, con una GPU NVIDIA RTX A6000 de 48 GB. En el clúster el archivo se llama `PL13'_SCN0001.las`, con apóstrofo, y el guion de lanzamiento lo quita al nombrar la salida, que queda como `PL13_SCN0001`. La entrada fue la matriz S que entrega la Unidad 1 para ese escaneo, generada en Khipu con los mismos parámetros

que la prueba sobre PL7. Su cadena de reducción fue de 8 239 904 puntos crudos a 8 020 796 tras el filtrado y a 803 099 representantes tras la voxelización, en 15,8 segundos. La matriz tiene 803 099 filas de 9 columnas. La Figura 3.11 resume los cinco pasos con los valores medidos. Cada paso se describe a continuación con su resultado en la prueba; los cortos llevan pseudocódigo y los largos, diagrama de flujo.

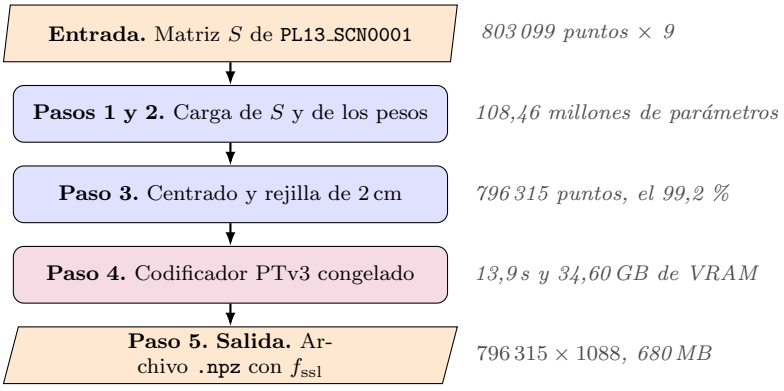


FIGURA 3.11: Flujo de la extracción con Sonata en PL13. Elaboración propia.

Paso 1. Carga de la matriz

El programa lee la matriz S que dejó la Unidad 1 y la separa en tres partes, la posición, el color y la normal. Son las tres entradas que espera Sonata. La matriz se lee completa porque, tras la voxelización, ya cabe en memoria. En la prueba se leen 803 099 filas en 0,3 segundos. La Figura 3.12 muestra esa entrada.

Algoritmo 4: Paso 1 del Módulo 3, carga de la matriz.

Entrada: archivo de la matriz $S \in \mathbb{R}^{N\times 9}$
Salida: *coord*, *color* y *normal*, cada una de $N \times 3$
$S \leftarrow$ leer el archivo
coord $\leftarrow$ columnas 1 a 3 de S
color $\leftarrow$ columnas 4 a 6 de S
normal $\leftarrow$ columnas 7 a 9 de S



FIGURA 3.12: Matriz S de PL13, entrada de Sonata. Elaboración propia.

Paso 2. Carga del modelo

El programa carga los pesos públicos de Sonata. Se descargan una sola vez, desde el nodo de acceso del clúster, y quedan guardados en disco, porque los nodos con GPU no tienen salida a internet. El modelo tiene 108,46 millones de parámetros y se pasa a la GPU en modo de inferencia, de modo que sus pesos no cambian. FlashAttention se desactiva porque no está instalada en el clúster, y el tamaño de parche del codificador se fija en 1024 puntos.

Algoritmo 5: Paso 2 del Módulo 3, carga del modelo.

Entrada: pesos públicos de Sonata guardados en disco
Salida: codificador PTv3_Φ listo en la GPU
$\Phi \leftarrow$ leer los pesos preentrenados sin conexión a internet
crear PTv3 con FlashAttention desactivada y parches de 1024 puntos
cargar Φ en PTv3, pasarlo a la GPU y fijarlo en modo de inferencia

Paso 3. Preparación

El programa aplica la rutina de preparación de Sonata. Primero traslada la nube, de modo que el centro de la caja en x e y y el punto más bajo en z quedan en el origen. Después la muestrea con una rejilla de 2 cm, según la Sección 2.2.9,

conservando un punto elegido al azar por celda, y guarda un índice inverso que dice a qué celda pertenece cada punto de entrada. Ese índice permite devolver más adelante los descriptores a cada punto de S. Por último divide el color entre 255. En la prueba quedan 796 315 de los 803 099 puntos, el 99,2 %. La diferencia es tan pequeña que a simple vista no se distingue, por eso este paso no lleva imagen.

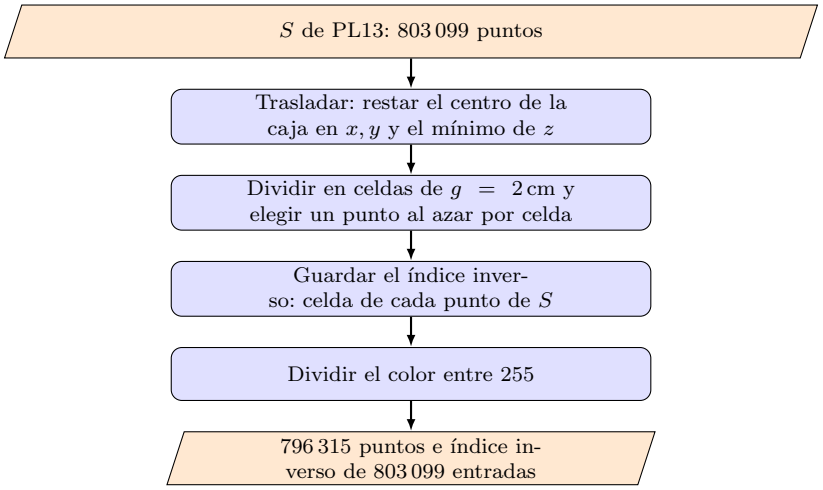


FIGURA 3.13: Preparación de la nube para Sonata. Elaboración propia.

Paso 4. Inferencia

El programa pasa la nube preparada por el codificador PTv3. Como el codificador es jerárquico, su salida queda a una resolución más gruesa que la entrada. El programa la devuelve a la resolución de la rejilla siguiendo la Sección 2.2.10, y así cada punto queda con un vector de 1088 dimensiones. En la prueba la inferencia tardó 13,9 segundos, procesó 57 397 puntos por segundo y usó un máximo de 34,60 GB de memoria de la GPU. La Figura 3.15 muestra el resultado. Los 1088 valores de cada punto se proyectan sobre sus tres componentes principales y se usan como color, de modo que un mismo color indica descriptores parecidos. Cada panel dibuja 100 000 puntos al azar, por eso la reducción de la Unidad 1 no se aprecia entre los dos primeros; se aprecia en la Figura 3.7.

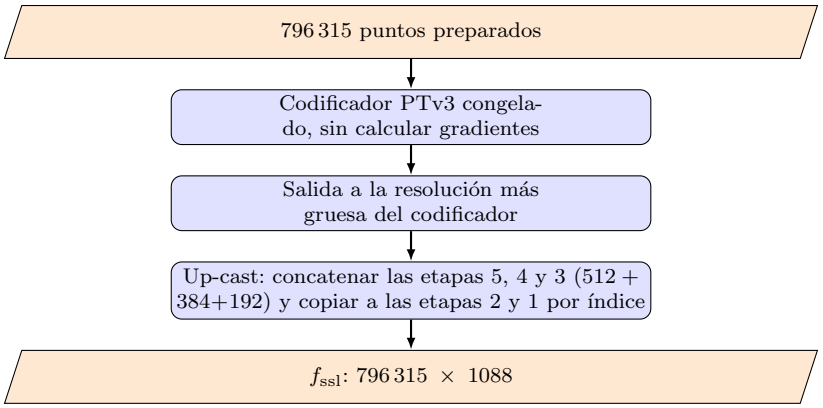


FIGURA 3.14: Inferencia de Sonata. Elaboración propia.



FIGURA 3.15: PL13 crudo, tras la Unidad 1 y con los descriptores de Sonata. Elaboración propia.

Paso 5. Escritura

El programa guarda en un archivo `.npz` todo lo que necesita la Unidad 2 para usar los descriptores sin volver a ejecutar Sonata. La Tabla 3.4 detalla su contenido. Los descriptores se guardan en precisión media para reducir el tamaño a la mitad. En la prueba el archivo ocupa 680 MB.

Algoritmo 6: Paso 5 del Módulo 3, escritura.

Entrada: f_{ssl}, *coord*, *inv* y g
Salida: archivo `.npz` en disco
convertir f_{ssl} a precisión media
guardar f_{ssl}, *coord*, *inv*, g y N en un archivo comprimido

Tabla 3.4: Archivo de descriptores de PL13. Elaboración propia.

Campo	Forma	Contenido
`feat`	$796\,315 \times 1088$	Descriptores f_{ssl} en precisión media
`coord`	$796\,315 \times 3$	Posición de cada punto de la rejilla
`inverse`	803 099	Celda de la rejilla de cada punto de S
`grid_size`	1 valor	Lado de la rejilla, 0,02 m
`n_original`	1 valor	Número de puntos de S, 803 099

Observables

El diseño declaró por adelantado qué debía medir esta corrida. La Tabla 3.5 compara lo esperado con lo medido en PL13. Los valores salen del registro que imprime `extraer_features.py` en la misma corrida, salvo el tamaño del archivo, que se lee del disco. El número de parámetros, 108,46 millones, lo imprime la función de carga de Sonata.

Tabla 3.5: Observables de Sonata en PL13: esperado y medido. Elaboración propia.

Observable	Para qué sirve	Esperado	Medido
Dimensión de f_{ssl}	Fija la entrada del Módulo 4	1088	1088
Puntos retenidos tras el muestreo por rejilla	Determina el tamaño del almacenamiento y la resolución efectiva	Por medir	796 315 de 803 099, el 99,2 %
VRAM pico	Verifica la cota analítica de memoria	Cota de 4,2 GB	34,60 GB
Tiempo de inferencia por sección	Sustituye a la estimación de costo	Por medir	13,9 s
Rendimiento en puntos por segundo	Permite extrapolar a otras secciones	Por medir	57 397
Tamaño del archivo de descriptores	Dimensiona el almacenamiento del proyecto	Por medir	680 MB

La memoria medida es unas ocho veces la cota analítica, que se calculó para una sección de 3×10^5 puntos y no para los 796 315 de esta prueba. La diferencia entre ambas no está explicada, y una causa posible es que, sin FlashAttention, la atención se guarde completa en memoria. Esa explicación no se ha verificado. Por

eso la medición reemplaza a la cota. PL13 usó el 72 % de los 48 GB de la GPU con 803 099 puntos. Si la memoria crece con el número de puntos, los escaneos más grandes del levantamiento podrían no caber en una sola pasada. Para ellos se prevé dividir la sección en bloques espaciales con un margen de solape y procesar cada bloque por separado.

Configuración de las rejillas

En la prueba, la resolución con que trabaja el codificador la fija la Unidad 1, porque la rejilla de 2 cm casi no elimina puntos tras el vóxel de 5 cm. Sonata se preentrenó con rejillas de 2 cm, de modo que aquí recibe una nube más gruesa que la de su entrenamiento. El tamaño de la rejilla de Sonata es una opción del programa, `--grid`, igual que el vóxel lo es de `preprocesar.py`.

Más adelante se harán pruebas con otras combinaciones de rejilla, por ejemplo con vóxeles de 2, 3 y 5 cm en la Unidad 1. En cada una se medirán los puntos retenidos, la memoria, el tiempo y la calidad de los descriptores. El objetivo es ver cómo la resolución afecta al agrupamiento del Módulo 4 y elegir el tamaño con datos, no por defecto.

Revisión de los descriptores

Los descriptores no traen etiquetas, así que su calidad todavía no se puede medir con una métrica de acierto. Se revisan con el visor de la Sección 3.3.3 y se miden con `visualizar_features.py`, de la carpeta `experimentacion`. Ambos programas aplican el mismo procedimiento. Restan a los descriptores la media de una muestra de 100 000 puntos elegidos con semilla 0 y los normalizan a largo 1. La Figura 3.16 los muestra de dos formas más, desde la misma cámara que la Figura 3.15, cuyo tercer panel ya muestra las componentes principales. La primera colorea cada

punto según su similitud coseno con un punto de consulta, marcado con un rombo negro, en rojo si se parece y en azul si no. La segunda agrupa los descriptores normalizados en ocho grupos con k-means, programado en NumPy, con 30 iteraciones y centros ajustados sobre esa misma muestra, sin etiquetas.

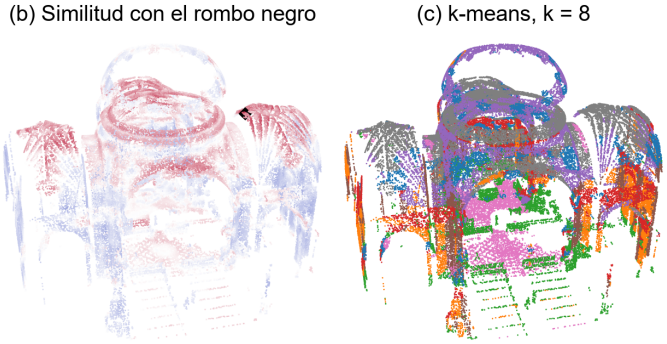


FIGURA 3.16: Similitud y k-means de los descriptores de PL13. Elaboración propia.

La medida de acierto es el *linear probing*, descrito en la Sección 2.4.7, que requiere puntos etiquetados. Mientras tanto, las tres primeras componentes principales explican el 25,3 % de la varianza, las diez primeras el 43,4 % y el rango efectivo es de 110,6 sobre 1088 dimensiones. Las varianzas se calculan por descomposición en valores singulares de esa muestra y el rango efectivo es la exponencial de la entropía de las varianzas normalizadas, $\exp\big(-\sum_k p_k \ln p_k\big)$. Vale 1 si todo queda en una sola dirección, lo que indicaría un colapso, y 1088 si se reparte por igual.

3.5.3 Punto de Fusión

El Módulo 4 (Sección 2.5.4) aún no se ha ejecutado. Concatena los dos descriptores según la Ec. (II.10) y deja igual todo lo posterior (Ec. (II.15), $M = 8$ vecinos), condición que hace interpretable la ablación.

3.6 Unidad 3. Entrenamiento, Calibración y Evaluación

La tercera unidad describe cómo se ejecuta el sistema en el tiempo, con qué procedimiento se obtienen las mediciones y bajo qué protocolo se repiten.

3.6.1 Procedimiento Experimental

El procedimiento se ejecuta en seis pasos reproducibles, correspondientes a los rótulos P0 a P5 de la Figura 3.1. Los dos primeros son insumo de la revisión crítica y fijan la línea base; los restantes corresponden a los objetivos específicos:

1. **P0. Recepción.** Nube TLS con al menos cuatro clases base anotadas en la sección de referencia, requisito mínimo de la tarea NCD.
2. **P1. Preparación y línea base.** Preprocesamiento de cada sección (centrado, SOR, voxelización a 5 cm y normales); ejecución de HyperNCD sin modificaciones con tres semillas distintas para fijar el punto de partida contra el que se compara el híbrido.
3. **P2. Inferencia congelada (OE1).** Ejecución del codificador sobre cada sección, con *cache* de características y *up-cast* para conciliar la resolución del *encoder* PTv3 con la rejilla de Minkowski, y escritura de f_{ssl} en disco.
4. **P3. Fusión y entrenamiento (OE1).** Sustitución del descriptor según la Ec. (II.10) y verificación de funcionamiento de extremo a extremo sobre una sección de control. Se comprueba que el mIoU de las clases conocidas no caiga respecto de la línea base.
5. **P4. Calibración (OE2).** Búsqueda por grilla acotada sobre α, M y ϵ en la sección de referencia; la configuración ganadora se congela y se aplica sin reajuste al resto de secciones.
6. **P5. Análisis (OE3).** Cómputo de ΔmIoU *novel* (Ec. (II.24)), de la brecha cerrada G (Ec. (II.25)) y del análisis de dominancia (Ec. (II.19)), más la ablación de las tres variantes del descriptor: la línea base $F_i = [f_{\text{sem}}; f_{\text{geo}}]$, el modelo híbrido $F_i = [f_{\text{sem}}; f_{\text{ssl}}]$ y la variante completa $F_i = [f_{\text{sem}}; f_{\text{geo}}; f_{\text{ssl}}]$ del Plan B (Sección 3.7). En las tres la afinidad geométrica del hipergrafo se calcula igual, de modo que la única diferencia entre ellas es el descriptor.

3.6.2 Protocolo de Repetición

Las secciones son independientes. Ninguna necesita información de otra, de modo que cada una constituye una medición separada y todas pueden ejecutarse de forma simultánea.

Cada configuración se ejecuta tres veces sobre cada sección y se reporta la media y la desviación entre repeticiones. La razón no es solo la variabilidad del entrenamiento. El submuestreo que precede al codificador elige al azar el punto representante de cada vóxel, por lo que dos ejecuciones sobre la misma sección no producen la misma f_{ssl}. Las tres repeticiones abarcan por ello el procedimiento completo, P2 y P3, y no solo el entrenamiento.

Como las secciones son independientes, pueden repartirse entre varias GPU. Con n_{gpu} unidades y n_{sec} secciones, la aceleración frente a la ejecución secuencial es

$$A = \min\left(n_{\text{gpu}},\, n_{\text{sec}}\right), \tag{III.1}$$

porque una sección no se reparte entre unidades. Con 23 secciones, más de 23 GPU no acortan el tiempo total.

3.6.3 Trazabilidad de la Métrica

La métrica se reporta en dos puntos, con *linear probing* sobre el descriptor y con mIoU al final, según la atribución por etapas de la Sección 2.4.10.

3.6.4 Verificación Empírica del Entorno de Cómputo

Antes de ejecutar el procedimiento fue necesario dejar operativo el entorno de HyperNCD en el clúster Khipu, porque su repositorio no declara todas sus dependencias. Lo difícil fue compilar MinkowskiEngine contra CUDA 11.8 con un compilador

GNU compatible. Con el entorno listo, una corrida llegó hasta la carga del primer lote.

3.7 Riesgos y Mitigación

(i) Disponibilidad de la nube anotada. NCD exige al menos las clases conocidas etiquetadas. Se asume la anotación manual de una sección de referencia con $\geq$4 clases base; si el levantamiento llega sin etiquetas, esa anotación se vuelve tarea previa del cronograma y condiciona el inicio de OE1.

(ii) Acceso y configuración del clúster. La configuración de GPU de Khipu ha cambiado respecto de la documentada previamente. Por ello el proyecto no estima tiempos de calendario. Verifica empíricamente el hardware asignado (Tabla B, Sección 3.6.4).

(iii) Sustitución frente a concatenación (Plan B). Si el reemplazo directo degrada el rendimiento, la variante es concatenar $F_i = [f_{\text{sem}}; f_{\text{geo}}; f_{\text{ssl}}]$, dejando que el módulo de prototipos pondere las tres fuentes. Cualquier resultado, sea mejora, paridad o degradación, documentado con ablación es reportable.

3.8 Conclusión del Marco Metodológico

Las tres unidades preparan la nube, extraen los dos descriptores y los evalúan aislando una sola variable, el descriptor que entra al módulo de prototipos. Así, cualquier diferencia en las métricas del capítulo siguiente se atribuye al reemplazo de f_{geo} por f_{ssl}.

REFERENCIAS BIBLIOGRÁFICAS

[1] Q. Wang and M.-K. Kim, "Applications of 3D point cloud data in the construction industry: A fifteen-year review from 2004 to 2018," *Advanced Engineering Informatics*, vol. 39, pp. 306–319, 2019.

[2] F. Matrone, A. Lingua, R. Pierdicca, E. S. Malinverni, M. Paolanti, E. Grilli, F. Remondino, A. Murtiyoso, and T. Landes, "A benchmark for large-scale heritage point cloud semantic segmentation," in *The International Archives of the Photogrammetry, Remote Sensing and Spatial Information Sciences*, vol. XLIII-B2-2020, 2020, pp. 1419–1426.

[3] R. Pierdicca, M. Paolanti, F. Matrone, M. Martini, C. Morbidoni, E. S. Malinverni, E. Frontoni, and A. M. Lingua, "Point cloud semantic segmentation using a deep learning framework for cultural heritage," *Remote Sensing*, vol. 12, no. 6, p. 1005, 2020.

[4] E. Grilli and F. Remondino, "Machine learning generalisation across different 3D architectural heritage," *ISPRS International Journal of Geo-Information*, vol. 9, no. 6, p. 379, 2020.

[5] M. A. Alvarado Ramos and P. O. Cuaresma Puclla, "Segmentación y optimización de nubes de puntos para la conservación del patrimonio arquitectónico peruano usando redes neuronales de grafos dinámicos," Master's thesis, Universidad de Ingeniería y Tecnología (UTEC), Lima, Perú, 2024.

[6] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, "Point transformer V3: Simpler, faster, stronger," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, arXiv:2312.10035.

[7] X. Wu, D. DeTone, D. Frost, T. Shen, C. Xie, N. Yang, J. Engel, R. Newcombe, H. Zhao, and J. Straub, "Sonata: Self-supervised learning of reliable point representations," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025, arXiv:2503.16429.

[8] X. Lai, J. Liu, L. Jiang, L. Wang, H. Zhao, S. Liu, X. Qi, and J. Jia, "Stratified transformer for 3D point cloud segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, arXiv:2203.14508.

[9] D. Robert, H. Raguet, and L. Landrieu, "Efficient 3D semantic segmentation

with superpoint transformer," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, arXiv:2306.08045.

[10] Y. Su, S. Chen, and Y.-G. Wang, "Balanced residual distillation learning for 3D point cloud class-incremental semantic segmentation," *arXiv preprint arXiv:2408.01356*, 2025.

[11] L. Riz, C. Saltori, E. Ricci, and F. Poiesi, "Novel class discovery for 3D point cloud semantic segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

[12] R. Xu, C. Zhang, H. Ren, and X. He, "Dual-level adaptive self-labeling for novel class discovery in point cloud segmentation," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2024.

[13] Z. Zhang, A. Wu, Y. Li, Y. Han, and J. Shen, "Geometric-aware hypergraph reasoning for novel class discovery in point cloud segmentation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026, arXiv:2606.07280.

ANEXOS

Anexo A. Configuración de *software*

El proyecto usa tres entornos. En Khipu, `sonata-env` (Python 3.10, PyTorch 2.5.0, CUDA 12.4, `spconv` y Pointcept) ejecuta Sonata, y `hyperncd-env` (Python 3.8, PyTorch 1.10.1, MinkowskiEngine, que declara CUDA 11.3 y se compiló con 11.8) ejecuta HyperNCD. En el equipo local, Python 3.12.7 con `laspy` 2.7.0, NumPy 2.3.5 y Open3D 0.20.0 ejecutó la prueba de la Unidad 1. El seguimiento de experimentos usa Weights & Biases.

Tabla A.1: Herramientas empleadas y módulo en que interviene cada una. Elaboración propia.

Herramienta	**Uso**	**Módulo**
`laspy`	Lectura por bloques del archivo LAS	1
Open3D	Filtrado SOR, voxelización y estimación de normales; reconstrucción de Poisson para exportar el sólido	1, 5
NumPy	Apilado de coordenadas y color, centrado, y almacenamiento de S y de f_{ssl} en disco	1, 3
PyTorch	Entrenamiento de la rama semántica y del módulo de prototipos; inferencia de Sonata	2, 3, 4
MinkowskiEngine	Rejilla dispersa y convoluciones de la MinkowskiUNet-34C	2
Pointcept, `spconv`	Codificador PTv3 de Sonata con sus pesos públicos	3
Código oficial de HyperNCD	Prototipos, hipergrafo y pseudo-etiquetas	2, 4
Weights & Biases	Registro de métricas por corrida	5
Clúster Khipu (Slurm)	Ejecución en CPU del Módulo 1 y en GPU de los Módulos 2 a 4	1–4
Plotly	Visor 3D del proyecto para revisar cada etapa y exportar las figuras	1, 3

Comandos en el clúster Khipu. Los guiones se lanzan desde la carpeta del repositorio, con los archivos LAS en `datos_crudos/`. La variable `REPO` al inicio de cada guion apunta a la copia del repositorio en el clúster y debe ajustarse al usuario.

```
# Unidad 1: todos los LAS, o solo los que calcen con el patron
sbatch preprocesar.sbatch
sbatch preprocesar.sbatch "PL13*"
# Cambiar parametros (voxel en metros, vecinos, umbral)
VOXEL=0.03 LAM=2.5 sbatch preprocesar.sbatch
# Rehacer los que ya existen
REHACER=1 sbatch preprocesar.sbatch
# Unidad 2, rama auto-supervisada: una seccion por trabajo
sbatch extraer_features.sbatch data/PL13_SCN0001.npy cache/PL13_feats.npz
# Medidas de los descriptores de Sonata (rango efectivo, PCA, k-means)
sbatch experimentacion/modulo3_sonata/visualizar_features.sbatch \
       PL13_SCN0001 PL13
# Visor 3D (en el equipo local, con los archivos copiados de Khipu)
python experimentacion/visor_3d.py --las PL13_SCN0001.las \
       --npy PL13_SCN0001.npy --json PL13_SCN0001.json \
       --feats PL13_feats.npz --marcar-sor --output visor_PL13.html
# Seguimiento
squeue -u $USER
tail -f logs/preproc_<id>.out
```

Anexo B. Nomenclatura

Cada símbolo de las fórmulas del marco teórico enlaza con su fila en esta tabla.

Símbolo	Significado	Valor
P	Nube de puntos del levantamiento	Catedral de Lima, TLS
x_i	Posición del punto i	Metros
x, y, z	Coordenadas de un punto, ya decodificadas	Metros
x'_i	Posición del punto i después del centrado	Metros
N	Número de puntos	610 637 en PL7 tras voxelizar
X, s_x, o_x	Entero almacenado, escala y desplazamiento de la cabecera LAS	Ec. (II.1)
c	Centro de la sección, restado al centrar las coordenadas	Guardado en los metadatos
q	Vecinos para SOR y normales	20
$\mathcal{N}_q(i)$	Vecindad de los q vecinos más cercanos del punto i	

Símbolo	Significado	Valor
$\bar{d}_i$	Distancia media del punto i a sus q vecinos	Ec. (II.3)
$\mu,\ \sigma$	Media y desviación estándar de $\bar{d}_i$ sobre la sección	
λ	Desviaciones toleradas en SOR	2
$v,\ \lvert v\rvert,\ \bar{x}_v$	Vóxel, número de puntos que contiene y su representante	Vóxel de 5 cm
$p_j,\ \bar{p}$	Vecinos de un punto y su centroide local	Ec. (II.6)
C	Matriz de covarianza de la vecindad	Ec. (II.7)
n_i	Normal unitaria del punto i	(n_x, n_y, n_z)
$R,\ G,\ B$	Color del punto: rojo, verde y azul	0 a 255
$\top$	Traspuesta de un vector o de una matriz	
S	Sección preprocesada: posición, color y normal	9 columnas por punto
$\mathcal{C}_k$	Conjunto de clases conocidas (etiquetadas)	Por sección
$\mathcal{C}_n$	Conjunto de clases novedosas (sin etiqueta)	Hasta 6 por sección
f_{sem}	Descriptor semántico (MinkowskiUNet-34C)	96 dimensiones
f_{geo}	Descriptor geométrico calculado a mano	6 dimensiones
$\lambda_1, \lambda_2, \lambda_3$	Autovalores de la covarianza de una región, de mayor a menor	
f_{ssl}	Descriptor auto-supervisado (Sonata, congelado)	1088 dimensiones
$[\,a;\,b\,]$	Concatenación: el vector b colocado a continuación del vector a	
F_i	Descriptor compuesto del punto i	1184 dimensiones
K	Número de prototipos	Igual al de clases
$z_{i,k}$	Puntuación del punto i para el prototipo k	Cualquier número real
$s_{i,k}$	Peso de pertenencia del punto i al prototipo k	Entre 0 y 1
S_p	Matriz de asignación suave, formada por los $s_{i,k}$	
P_k	Prototipo k	Ec. (II.12)
$\mathcal{H}$	Hipergrafo de regiones	
$F^{(i)}$	Descriptor de la región i, promedio de los F_i de sus puntos	
$g^{(i)}$	Rasgo geométrico de la región i	Su f_{geo}
S_g	Afinidad geométrica entre regiones	Núcleo gaussiano
S_s	Afinidad semántica entre regiones	Coseno
S_b	Afinidad combinada	Ec. (II.15)
α	Balance geométrico-semántico en S_b	$[0, 1]$; 0,5 en la implementación

Símbolo	Significado	Valor
τ	Temperatura del núcleo gaussiano; un valor pequeño solo considera afines a las regiones muy próximas	0,1
M	Vecinos por hiperarista	8
ϵ	Suavizado de pseudo-etiquetas	0,15 en el artículo
$\hat{y}_i$	Etiqueta asignada al punto i	Clase conocida o novedosa
c (subíndice)	Una clase del conjunto $\mathcal{S}$	
$\mathcal{S}$	Conjunto de clases sobre el que se calcula una métrica	$\mathcal{C}_k$, $\mathcal{C}_n$ o $\mathcal{C}$
$\mathcal{C}$	Todas las clases	$\mathcal{C}_k \cup \mathcal{C}_n$
TP_c, FP_c, FN_c	Verdaderos positivos, falsos positivos y falsos negativos de la clase c	Número de puntos
IoU_c	Intersección sobre unión de la clase c	Ec. (II.16)
$\mathrm{mIoU}(\mathcal{S})$	Promedio del IoU sobre el conjunto de clases $\mathcal{S}$	Ec. (II.17)
c^*	Clase dominante del mIoU	Ec. (II.19)
σ^*	Emparejamiento óptimo entre grupos descubiertos y clases reales	Ec. (II.20)
σ, $\mathfrak{S}_{\lvert\mathcal{C}_n\rvert}$	Una correspondencia entre grupos descubiertos y clases novedosas, y el conjunto de todas ellas	
y	Etiqueta real de un punto	
ARI	Índice de Rand ajustado	Ec. (II.21)
n	Número total de puntos comparados	
n_{ij}, a_i, b_j	Puntos comunes entre la agrupación i y la clase j, y sus marginales	Ec. (II.21)
NMI	Información mutua normalizada	Ec. (II.22)
U, V	Las dos agrupaciones que se comparan: la descubierta y la real	
I, H	Información mutua y entropía	Ec. (II.22)
W, b	Pesos y sesgo del clasificador lineal del *linear probing*	Ec. (II.23)
$\Delta\mathrm{mIoU}^{\mathrm{novel}}$	Ganancia absoluta en clases novedosas	Ec. (II.24)
mIoU_b^a	mIoU del modelo b (*hib*: híbrido; *base*: línea base) sobre las clases a (*novel*: novedosas; *known*: conocidas)	
G	Brecha cerrada	Ec. (II.25)

Índices y operadores

Símbolo	Significado	Valor
$i,\ j,\ k$	Índices: i y j recorren puntos, vecinos o regiones; k, prototipos	
$\sum$	Suma de los términos que indica el subíndice	
mín, máx	Mínimo y máximo, tomados eje por eje	
arg máx	Elemento para el que la expresión toma su valor más alto	
exp	Función exponencial	
$\|\cdot\|$	Longitud de un vector (norma euclídea)	
$\langle\cdot,\cdot\rangle$	Producto escalar de dos vectores	
$\|\cdot\|$	Número de elementos de un conjunto; aplicado a un número, su valor absoluto	
$\in$	Pertenece a	
∂	Derivada parcial: cuánto cambia una cantidad al variar otra	
$\forall$	Para todo	
$\binom{m}{2}$	Número de pares que se pueden formar con m elementos	$m(m-1)/2$

Anexo C. Arquitectura del *encoder* de Sonata

Sonata es un Point Transformer V3 (PTv3) usado solo como codificador: cinco etapas jerárquicas, sin decodificador, con 108,46 M de parámetros. La configuración que carga el programa coincide con la de la Figura C.1: el recuento de parámetros de la Tabla C.1, hecho capa por capa, da exactamente el total que el modelo declara al cargarse.

El 94,3 % de los parámetros está en las etapas 4 y 5. Los pesos ocupan 0,43 GB en precisión simple. La memoria de vídeo medida en la inferencia, 34,60 GB para PL13, se debe a las activaciones y no a los pesos (Sección 3.5.2).

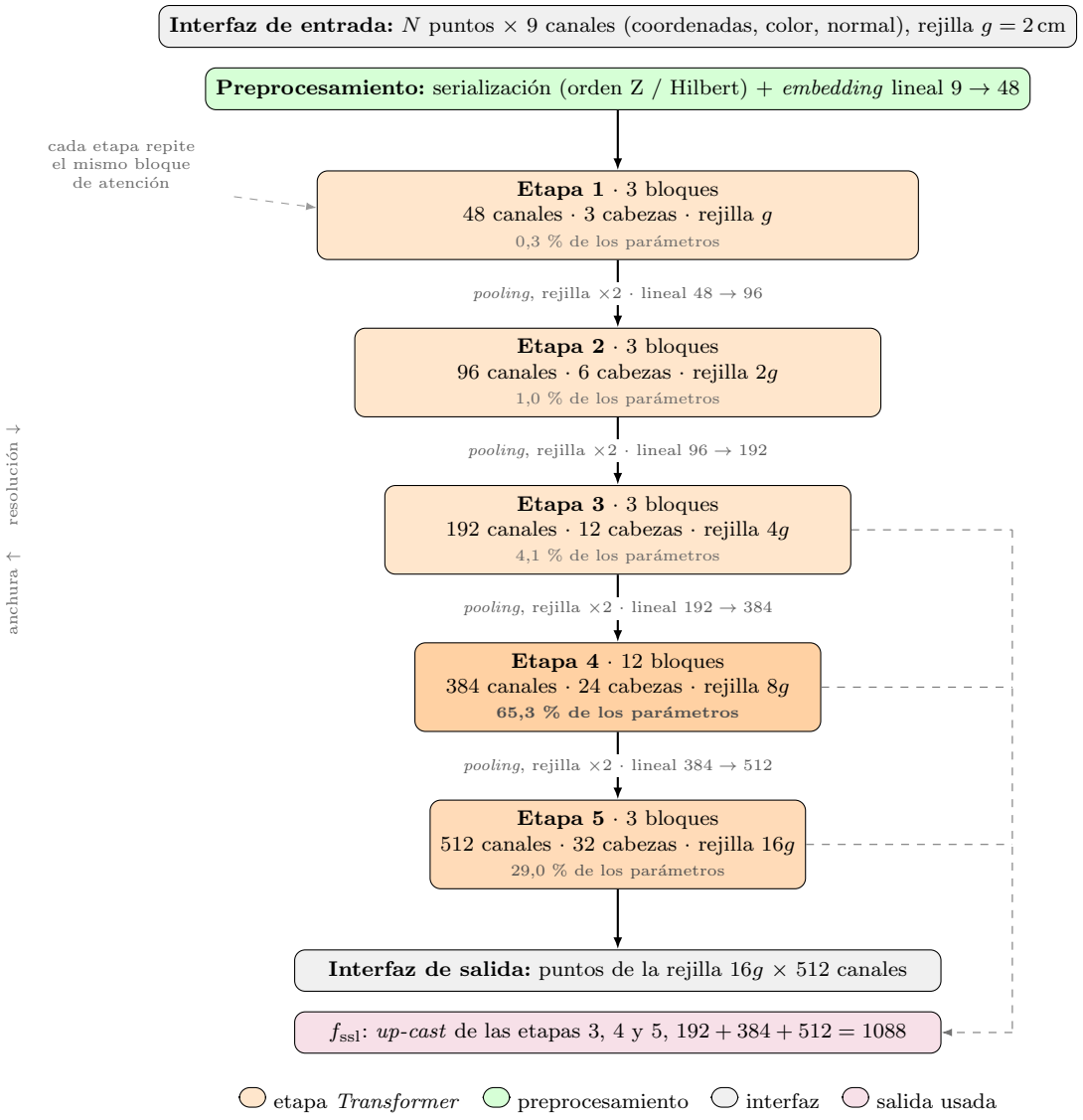


FIGURA C.1: Arquitectura del *encoder* de Sonata: cinco etapas sin decodificador. Las líneas discontinuas marcan las etapas que forman f_{ssl}. Elaboración propia.

Tabla C.1: Distribución de parámetros del *encoder* de Sonata por etapa.

Etapa	**Canales**	**Bloques**	**Parámetros**	**%**
1	48	3	278 928	0,3
2	96	3	1 110 816	1,0
3	192	3	4 433 472	4,1
4	384	12	70 857 216	**65,3**
5	512	3	31 483 392	29,0
Embedding y submuestreo			297 504	0,3
Total			**108 461 328**	100,0